



Universidade do Minho

Escola de Engenharia

Programação Imperativa

LETI :: 2022/23

Setembro 2022

© António Esteves

© Slides adaptados de:

CSC230 - C and Software Tools, Computer Science Faculty, Duke University, USA

BE5B99CPL – C Programming Language, Jan Faigl, Czech Technical University in Prague

Bibliografia essencial:

C Programming: A Modern Approach, capítulos 2 e 3

1. Introdução ao Paradigma de Programação Imperativa com a Linguagem C

- Contextualização da linguagem C
- Características mais relevantes da linguagem C
- Estrutura dum programa em C
- Tipos de dados primários do C
`int, float, double, char, void`
- Programar em C
diretrizes, caracteres válidos, identificadores, compilação, funções
- Valores e variáveis
tipos de literais, declaração e atribuição e armazenamento em memória de variáveis
- Operadores e expressões
- Entrada e saída normalizadas
leitura e escrita de um caractere, escrita formatada com `printf`, leitura formatada com `scanf`
- C89 e C99

- **Contextualização da linguagem C**
- Caraterísticas mais relevantes da linguagem C
- Estrutura dum programa em C
- Tipos de dados primários do C
- Programar em C
- Valores e variáveis
- Operadores e expressões
- Entrada e saída normalizadas
- C89 e C99

Contextualização: criadores do C



Ken Thompson

Dennis Ritchie

AT&T Bell Labs, 1969-1972



Contextualização: quais eram os requisitos iniciais do C?

- O **código binário** após compilação deveria **ocupar pouco espaço** em memória → **estava-se em 1972!**
- O código do próprio sistema operativo e das aplicações devia
 - ser **compacto**
 - ter **bom desempenho**
 - correr em **diversas plataformas**
- Não havia preocupação em que o código:
 - tivesse execução segura e fiável
 - fosse fácil de manter
 - incluísse uma componente de testes

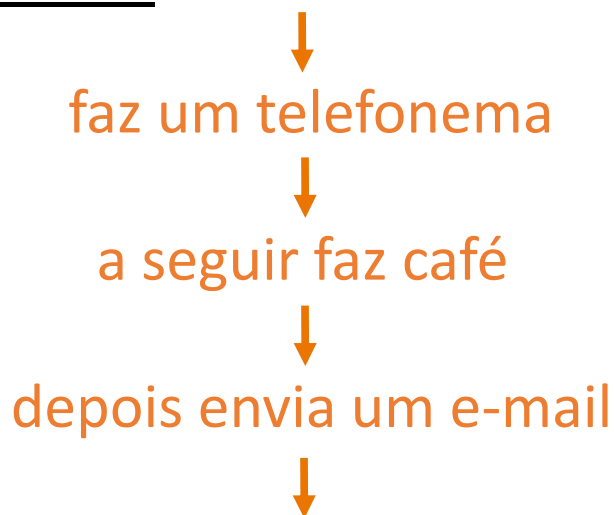
Contextualização: Porquê utilizar C?

- Funciona como uma “linguagem **assembly portátil**” de **mais alto nível**
- É útil para ...
 - Desenvolver **sistemas operativos e embebidos**
 - **Otimizar rotinas** de outras linguagens: **desempenho**, **tamanho** ou previsibilidade
- *Software* importante escrito integralmente em C:
 - **UNIX** e **Linux** -- núcleo e maioria dos utilitários
 - NetApp Data ONTAP -- *software* mais comum para gerir armazenamento de dados
 - **Compilador/interpretador** de Python, Perl, PHP, Java*, Ruby*
 - Uma imensidão de aplicações:
 - ❖ http://en.wikipedia.org/wiki/Category:Free_software_programmed_in_C

(*) Com algum código C++

Contextualização: programação imperativa

- Descreve-se a computação como uma **sequência de instruções**, ou comandos, a executar pelo computador
- As instruções **mudam o estado** do programa
- **estado** $\Leftrightarrow$ definido pelos **valores das variáveis**
- É análoga ao comportamento imperativo das línguas naturais, em que se dão ordens:



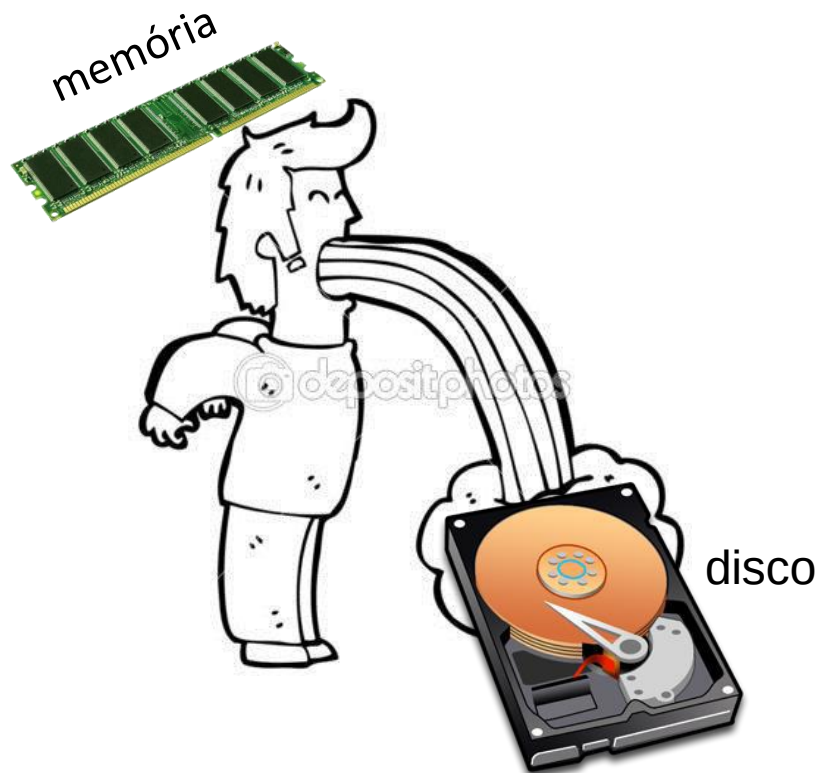
Tarefa: Guardar uma lista de coordenadas (*lat, long*) em disco a partir da memória

C

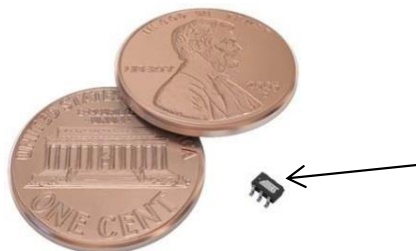
Maioria das linguagens HLL

- Escolher o formato do ficheiro
- Criar uma rotina para ler e analisar os dados da memória
- Criar uma rotina para serializar os dados a escrever em ficheiro
- Avaliar o desempenho caso isso seja relevante

- Ler de memória e escrever diretamente em ficheiro



Tarefa: Colocar um LED a piscar



Microcontrolador Atmel ATTINY4: um pequeno computador com CPU, RAM e memória flash. 1024 bytes de memória flash e 32 bytes de RAM.

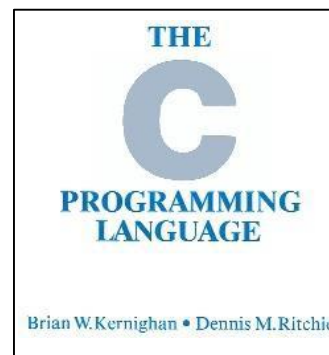
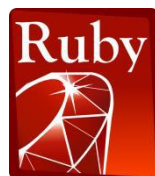
```
led = 0
enquanto (verdade):
    led = Negacao(led)
    DefinirLed(led)
    Esperar 1 seg
```

Linguagem	Tamanho do executável	Tamanho do ambiente de execução (excluindo bibliotecas)	Tamanho Total	RAM usada
Java	410 B	13 MB (java + libjvm)	13 MB	14 MB
Python	60 B (código fonte)	2.9 MB	2.9 MB	5.4 MB
C	8376 B	Nenhum	8376 B	352 kB
Embedded C (Arduino)	838 B	Nenhum	838 B	~16 B

Max: 1024 B

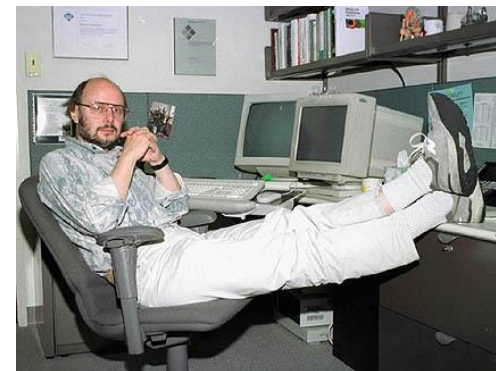
Max: 32 B

Contextualização: C vs. outras linguagens



Linguagens de alto nível populares	C
Para desenvolver aplicações	Para desenvolver sistemas ... e aplicações ↳ há uns anos eram a mesma coisa!
Os recursos do computador não são diretamente acedidos pelo programador	O programador tem acesso quase direto ao hardware
Favorecem práticas de programação que evitam comportamentos indesejados e que reduzem o impacto dos erros	A filosofia da linguagem é confiar no programador ; erros subtis podem ter efeitos enormes
Funcionam como por magia! Um exemplo: o mecanismo de garbage collection	<u>Nada</u> acontece sem a ordem do programador
O código pode ser executado numa máquina virtual ou num interpretador	O código fonte é compilado para código de máquina nativo da arquitetura alvo

- Originalmente era designado como “C com classes”
- Todos os programas em C são programas válidos em C++, dado que o C++ é uma extensão do C



Bjarne Stroustrup criou o C++
em 1979 nos Bell Labs

- Inclui muitas características comuns ao Java:
 - **Classes** – classes abstratas e funções virtuais
 - **Overloading** (sobrecarregar) dos operadores
 - **Herança**, incluindo herança múltipla
 - **Tratamento de exceções**

C

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, const char* argv[]) {
    int i;

    printf("Hello, world.\n");

    for (i=0; i<3; i++) {
        printf("%d\n", i);
    }

    return EXIT_SUCCESS;
}
```

```
$ gcc -o thing thing.c && ./thing
Hello, world.
0
1
2
```

Java

```
class Thing {
    static public void main (String[] args) {
        int i;

        System.out.printf("Hello, world.\n");

        for (i=0; i<3; i++) {
            System.out.printf("%d\n", i);
        }
    }
}
```

```
$ javac Thing.java && java Thing
Hello, world.
0
1
2
```

- Contextualização da linguagem C
- **Caraterísticas mais relevantes da linguagem C**
- Estrutura dum programa em C
- Tipos de dados primários do C
- Programar em C
- Valores e variáveis
- Operadores e expressões
- Entrada e saída normalizadas
- C89 e C99

Caraterísticas mais relevantes da linguagem C₍₁₎

- É a linguagem em que foi escrito o UNIX/**Linux**
- É uma linguagem orientada às **funções**
... não suporta classes
- Permite acesso de **baixo nível** à memória
- É fácil de converter para código máquina
- Não precisa de ambiente de execução
... a não ser o Sistema Operativo (SO)
- Apesar do nível médio/baixo é multiplataforma

Porquê aprender C em 2022?

Programar/aprender sistemas operativos,
programar sistemas embebidos/microcontroladores/outro *hardware*,
é a base de várias linguagens de mais alto nível, ...

Caraterísticas mais relevantes da linguagem C₍₂₎

- Utiliza **poucas palavras chave** → `if, else, for, while, do, return, switch, case, break, continue, struct, union, int, ...`
- Suporta tipos de **dados compostos** → `struct, union`
- **Apontador** → endereço de (memória onde está guardada) uma variável)
- **Bibliotecas externas** normalizadas → entrada/saída, funções matemáticas, acesso à memória, outras facilidades
- Utilizada em conjunto com um **pré-processador** de diretivas
→ `#define, #include, ...`

Caraterísticas mais relevantes da linguagem C₍₃₎

- O código C é
 - **rápido ?!** como é compilado, é por natureza executado mais rapidamente que código equivalente interpretado
 - **Compacto** ⇔ pouco verboso
 - de uso **genérico**
 - maioritariamente **independente da arquitetura alvo**
- O C é utilizado para programar sistemas muito diversos
 - compiladores
 - sistemas operativos
 - sistemas embebidos, micro-controladores, ...
 - computação genérica e computação de elevado desempenho

O C é uma linguagem ...

- com **verificação estática** do **tipo** das variáveis → os tipos são identificados pelo compilador
 - ❖ vs. dinâmica → os tipos só são conhecidos durante a execução
- com **fraca segurança nos tipos** das variáveis → permite atribuir valores de tipo errado às variáveis, contribuindo p/ falhas na execução
- incentiva a **programação estruturada**
 - ❖ uso extensivo de construtores de controlo do fluxo, de repetição e de funções
 - ❖ melhora a clareza e qualidade; reduz o tempo de desenvolvimento
- **imperativa**

- Contextualização da linguagem C
- Características mais relevantes da linguagem C
- **Estrutura dum programa em C**
- Tipos de dados primários do C
- Programar em C
- Valores e variáveis
- Operadores e expressões
- Entrada e saída normalizadas
- C89 e C99

"Hello world" em C



Universidade do Minho
Escola de Engenharia

```
#include <stdio.h>
#include <stdlib.h>
```

Ficheiros header
com declarações
de funções da
biblioteca
normalizada do C

```
int main(int argc, const char* argv[])
{
```

Ponto de entrada
no programa, com
argumentos

```
    int i;
```

```
    printf("Hello, world.\n");
```

```
    return EXIT_SUCCESS;
```

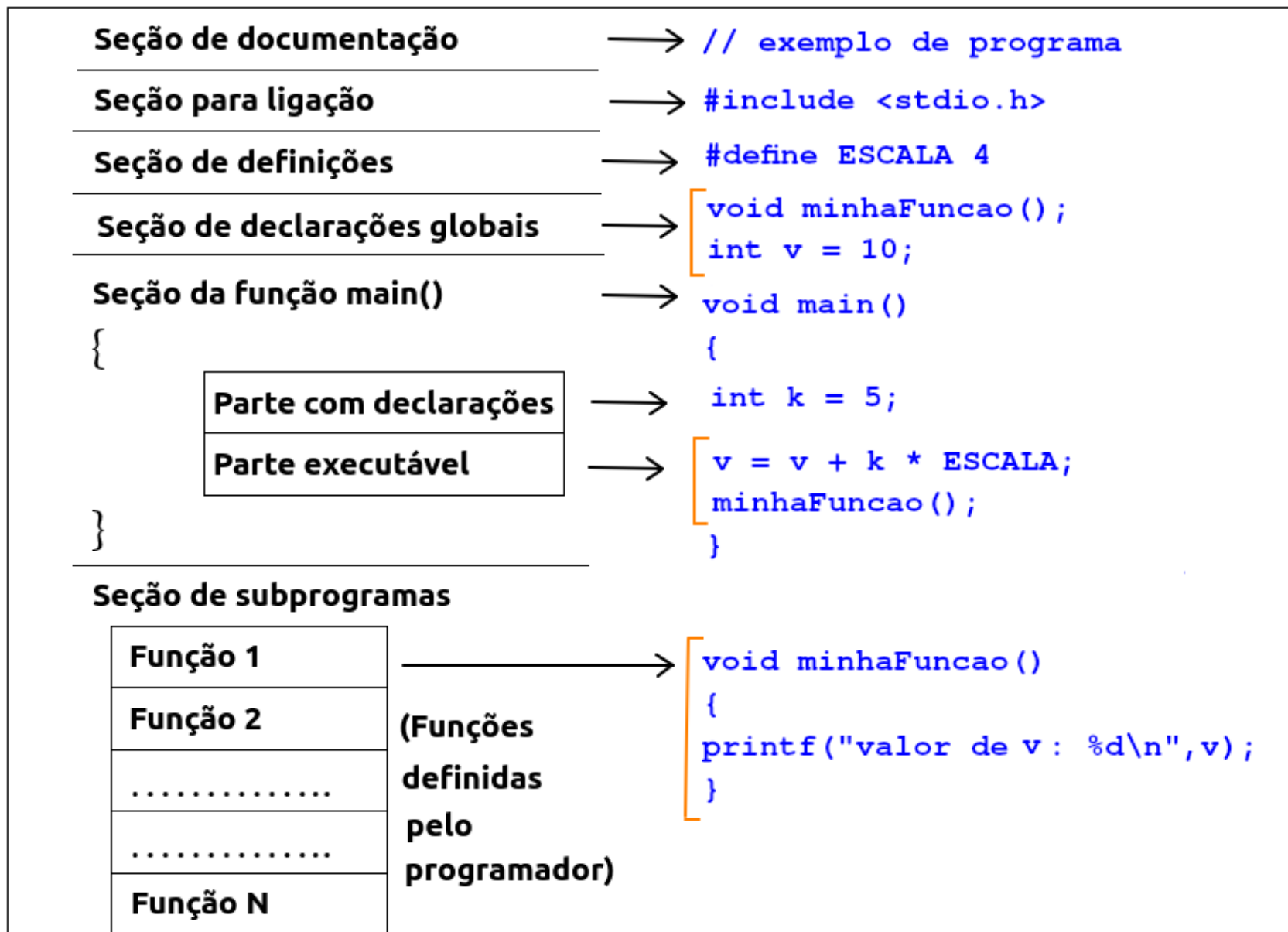
```
}
```

Sai do programa e
sinaliza conclusão bem
sucedida

Função da biblioteca
normalizada do C, usando
uma mensagem como
argumento

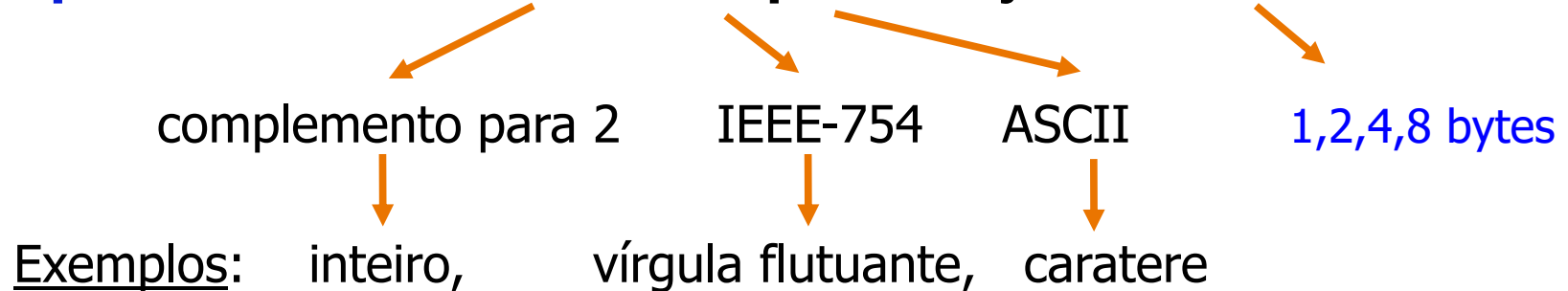
```
$ gcc -Wall -std=c99 hello.c -o hello
$ ./hello
Hello, world.
```

Estrutura dum programa em C



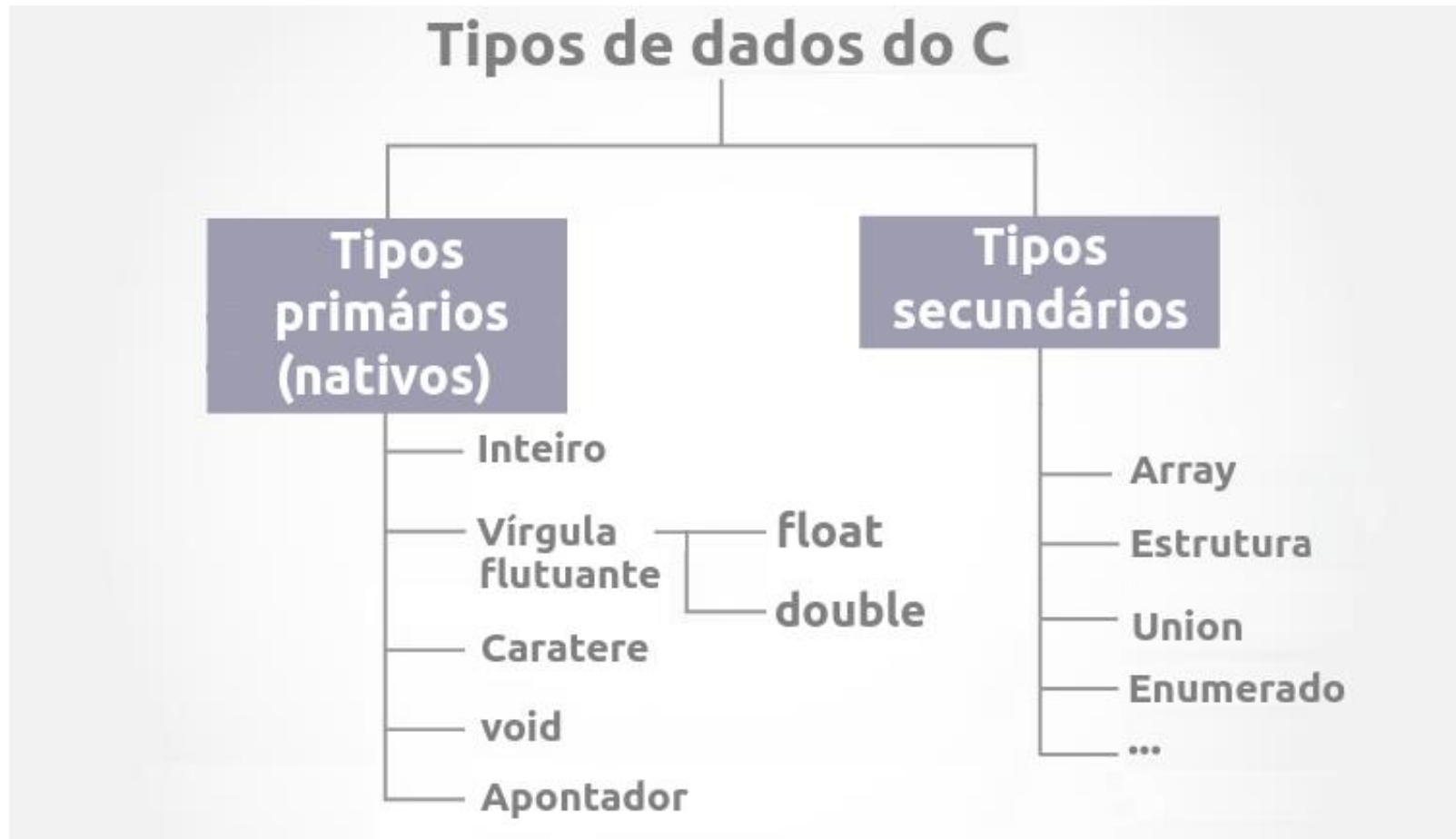
- Contextualização da linguagem C
- Características mais relevantes da linguagem C
- Estrutura dum programa em C
- **Tipos de dados primários do C**
- Programar em C
- Valores e variáveis
- Operadores e expressões
- Entrada e saída normalizadas
- C89 e C99

tipo de dados $\equiv$ **formato de representação** + **tamanho**



Os tipos de dados utilizam-se:

- Na declaração e definição de **variáveis**
- Nos **argumentos** das funções
- No **valor devolvido** pelas funções



Tipos de dados primários

DECLARAÇÃO DE VARIÁVEL DO TIPO `int`:

```
int contador;
```

ATRIBUIÇÃO DE UM VALOR À VARIÁVEL:

```
contador = 5;
```

DECLARAÇÃO DE VARIÁVEL DO TIPO `float`:

```
float distancia;
```

ATRIBUIÇÃO DE UM VALOR À VARIÁVEL:

```
distancia = 4.75;
```

DECLARAÇÃO DE VARIÁVEL DO TIPO `char`:

```
char letra;
```

ATRIBUIÇÃO DE UM VALOR À VARIÁVEL:

```
letra = 'w';
```

FUNÇÃO COM VALOR DEVOLVIDO DO TIPO `void`:

```
void printMensagem(char *);
```

FUNÇÃO COM ARGUMENTO DO TIPO `void`:

```
double randomNumber(void);
```


Qualificadores do C

- Alteram o significado dos tipos de dados
 - ↳ produzem novos tipos de dados
- Qualificadores de tamanho → **long**
 - **short**
- Qualificadores de sinal → **signed**
 - **unsigned**

(só para inteiros)
- Qualificador de constante → **const**

<pre>long double lf; long long int lli; long int li; short int si;</pre>	<pre>unsigned int peso; unsigned short int idade;</pre>
<pre>const float fConversao = 3.7; const int fEscala = 5; fEscala = 6; // ERRO!!</pre>	<pre>01-constante.c</pre>

- Contextualização da linguagem C
- Características mais relevantes da linguagem C
- Estrutura dum programa em C
- Tipos de dados primários do C
- **Programar em C**
- Valores e variáveis
- Operadores e expressões
- Entrada e saída normalizadas
- C89 e C99

- Regras universalmente recomendadas:
1. **clareza** e **consistência** são importantes
 2. utilizar **indentação**, **espaços em branco** e **comentários** é útil
 3. É útil manter o mesmo **estilo nos nomes** das funções, variáveis e constantes

- **Comentários no início de cada ficheiro** fonte:
 - Nome do ficheiro
 - Nome e número do(s) autor(es)
 - Data
 - Resumo dos objetivos do programa ou do módulo
- **Comentários** a documentar cada **função**:
 - **Descrição** resumida dos objetivos da função
 - **Argumentos**: nome e breve explicação de cada um
 - **Saídas**: valor devolvido e efeitos colaterais (por exemplo variáveis externas alteradas utilizando o seu endereço)

- Não usar **variáveis globais** a não ser em casos excepcionais
- No caso de usar variáveis globais, indicar o seu objetivo em comentário
- **Constantes**
 - Utilizar **#define** para definir o valor de constantes sempre que a sua importância no programa o justifique

➤ Indentação

- Usar de forma **consistente** o mesmo tipo de indentação:
 - um número fixo de espaços (ex: 4) ou
 - uma tabulação com um tamanho fixo
- **O que indentar:**
 - As declarações e instruções dentro de uma **função**
 - As instruções dentro de numa **estrutura de controlo**
 - As declarações dos membros duma **struct** ou **union**

➤ Colocação das **chavetas** { }

- **Funções:** a chaveta de abertura { é colocado no início da linha seguinte
- **Em tudo o resto:** a chaveta de abertura { é colocada no fim da estrutura de controlo ou da `struct / union`
- A chaveta de fecho } é colocada sempre no início da linha seguinte

➤ Declarações e instruções

- **uma** por linha

TPC Carateres válidos no código fonte C



Universidade do Minho
Escola de Engenharia

- Letras minúsculas, letras maiúsculas, carateres numéricos, símbolos e separadores
 - a-z A-Z 0-9
 - ! " # % & ' () * + , - . / : ; < = > ? [\] ^ _ { | } ~
 - espaço em branco, tabulação, mudança de linha
 - Sequências de *escape* para escrever símbolos especiais: ' " ? \ \ ' para escrever a plica, \ " para a plica dupla, \ ? para ?, \\ para \
 - Sequências de *escape* para escrever valores numéricos em fragmentos de texto:
 - \o, \o..o, onde o é um dígito na base octal (0..7)
 - \xh, \xh..h, onde h é um dígito na base hexadecimal (0..9 a..f)
- ```
int i = 'a';
int h = 0x61;
int o = 0141;
printf("i: %i h: %i o: %i c: %c\n", i, h, o, i);
printf("oct: \141 hex: \x61\n");
```
- \0 – caractere reservado para definir o fim duma *string* (valor é 0)

ASCII – código normalizado  
Americano para troca de informação  
01-esqdho.c



- Os **identificadores** são **nomes** de variáveis, nomes de tipos definidos pelo utilizador e nomes de funções
  - Regras para definir **identificadores**
    - Carateres **a-z, A-Z, 0-9, \_**
    - O primeiro caratere não pode ser um algarismo
    - Diferenciação entre maiúsculas e minúsculas (*case sensitive*)
    - Não há limite para o comprimento dum identificador
  - **Palavras-chave** (proibidas como identificador):  
**auto break case char const continue default do double else enum  
extern float for goto if int long register return short signed sizeof  
static struct switch typedef union unsigned void volatile while**
- O C99 introduziu:
- inline, restrict, \_Bool, \_Complex, \_Imaginary**
- O C11 adicionou:
- \_Alignas, \_Alignof, \_Atomic, \_Generic, \_Static\_assert, \_Thread\_local, \_Noreturn**

# Como escrever um programa em C <sup>(1)</sup>

- O código dum programa C é guardado em **ficheiros de texto**
  - Ficheiros **header**, normalmente com extensão **.h**
  - Ficheiros **fonte**, habitualmente com extensão **.c**

---

- Ficheiros **header**, ao conterem um conjunto de **declarações** e **definições** de funções e tipos de dados, permitem:
  - **Organização** do código fonte em vários ficheiros (**módulos**) e bibliotecas, permitindo **gerir melhor a complexidade** dos programas
  - **Modularidade**: os ficheiros *header* declaram a **interface** que é visível pelos outros ficheiros fonte

*Na prática é uma lista de tipos de dados e de funções, incluindo os seus argumentos e o tipo devolvido mas sem especificar a sua implementação*
  - **Reutilização**

Para se poder usar as funções contidas numa biblioteca ou em outro ficheiro binário, só é necessário conhecer a sua interface declarada num ficheiro *header*

# Como escrever um programa em C<sub>(2)</sub>

- Cada **programa** executável tem que incluir **pelo menos uma função** e o nome dessa função obrigatória tem que ser **main()**
- A **execução** do programa **começa no início da função main()**
- Um exemplo:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
 printf("Gosto de programar em C!\n");
```

```
 return 0;
```

```
}
```

- A estrutura da função **main()** é a apresentada acima

# Um programa simples em C

```
#include <stdio.h>
```

01-programa.c

```
int main(void)
```

```
{
```

```
 printf("Gosto de programar em C!\n");
```

```
 return 0;
```

```
}
```

- Os ficheiros **fonte** são **traduzidos** pelo **compilador** para ficheiros **objeto** → normalmente com extensão **.o** ou **.obj**

*Um **ficheiro objeto** contém endereços relativos e chamada de funções, mas a implementação dessas funções pode estar noutro ficheiro*

- O programa **executável** é criado pelo **linker** a partir dos ficheiros objeto

# Como gerar um executável a partir do programa C<sub>(1)</sub>

código fonte

```
#define N 3
a=c+b*N;
```

pré-processamento

código fonte expandido

```
a=c+b*3;
```

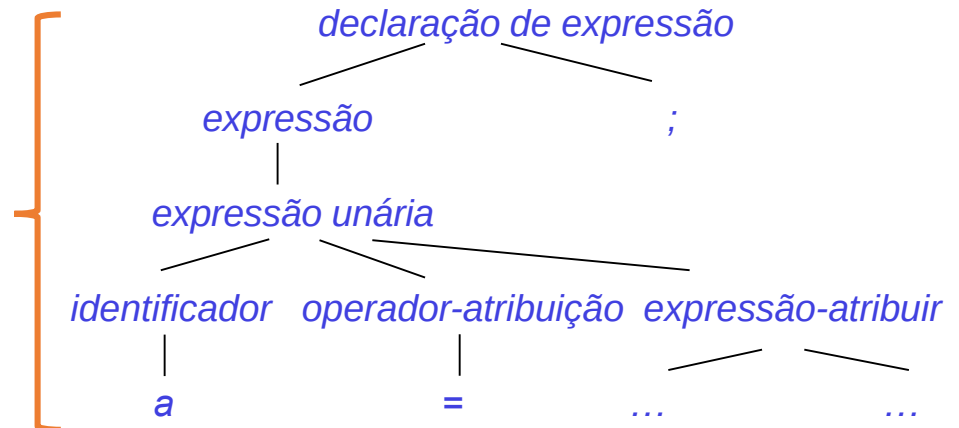
análise léxica

entidades (*tokens*)

```
a, =, c, +, b, *, 3, ;
```

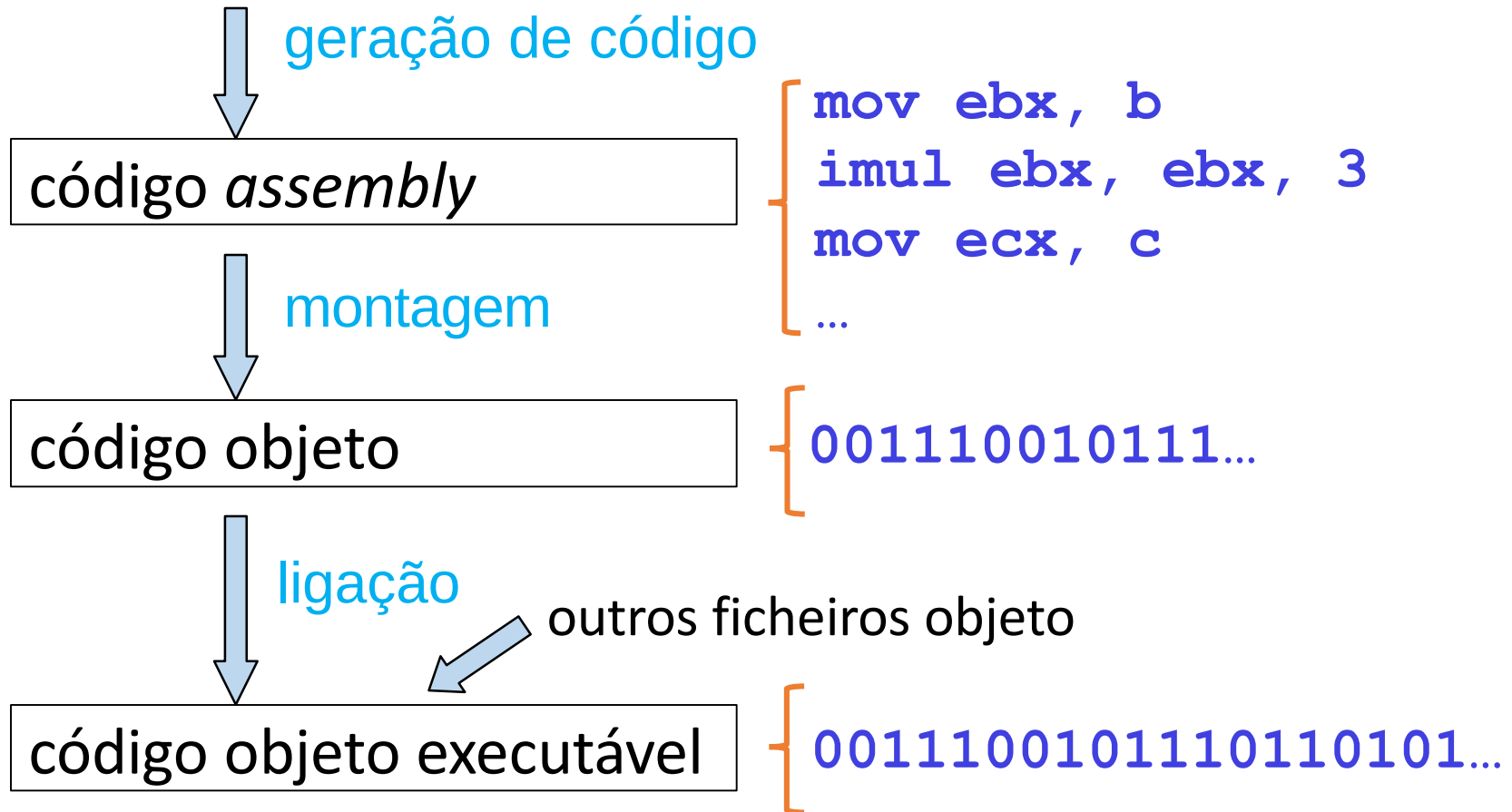
análise semântica

árvore de *parsing*



geração de código

# Como gerar um executável a partir do programa C<sub>(2)</sub>



# Como utilizar o compilador gcc

- O **gcc** é um compilador de código aberto disponível para a maioria das plataformas
- Na **linha de comandos** do **terminal** escrevemos:  

```
gcc -Wall -std=c99 01-programa.c
```

onde **01-programa.c** *contém* o código fonte do programa

  - ➡ O comando cria um executável **a.out**
- **-std=c99** indica que o código pode conter elementos da norma C99
- **-Wall** ativa todas as mensagens de aviso durante a compilação

|                       |                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------|
| <b>-o prog</b>        | Colocar o resultado da compilação <u>ou</u> da montagem <u>ou</u> da ligação no ficheiro <b>prog</b> |
| <b>-std=c99</b>       | Permitir características da norma C99                                                                |
| <b>-Wall</b>          | Ativar todos os avisos durante a compilação                                                          |
| <b>-E</b>             | Apenas faz o pré-processamento do código fonte → expande as macros e as constantes                   |
| <b>-S</b>             | Pré-processa e compila o código fonte<br>→ gera ficheiro(s) <i>assembly</i>                          |
| <b>-c</b>             | Compila e faz a montagem do código fonte mas não faz ligação → gera ficheiro(s) objeto               |
| <b>--version , -v</b> | Mostrar a versão do <b>gcc</b>                                                                       |
| <b>-g</b>             | Gerar código máquina com informação necessária para o <b>depurar</b> através do <b>gdb</b>           |



|                       |                                                                                                                                                     |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>-O0 , -O2, -O3</b> | Nível de otimização ( <b>O3</b> otimiza mais)                                                                                                       |
| <b>-lsufixo</b>       | Efetuar a ligação com a <b>biblioteca</b> de nome <b>libsufixo</b>                                                                                  |
| <b>-ldir</b>          | Adiciona a pasta <b>dir</b> à lista de pastas onde o compilador vai procurar os ficheiros <i>header</i> ( <b>i</b> maiúsculo de “ <b>In</b> cluir”) |
| <b>-Ldir</b>          | Adiciona a pasta <b>dir</b> à lista de pastas onde o compilador vai procurar os ficheiro objeto das bibliotecas                                     |

# Demo: compilação e execução de 01-programa.c



Universidade do Minho  
Escola de Engenharia

- O ficheiro fonte **01-programa.c** é compilado, por um compilador como **clang** ou **gcc**, gerando um ficheiro executável:  
**gcc 01-programa.c**
- Se não especificarmos o nome do ficheiro de saída da compilação, o seu nome será **a.out**
- Este ficheiro pode ser executado, utilizando o comando no terminal do **sistema operativo** (**SO**):  
**./a.out**
- O programa escreve no **terminal** o argumento da função **printf()**  
**./a.out**

Gosto de programar em C!

# Estrutura do código fonte: Exemplo comentado



## ➤ Ficheiro **01-programa.c** com código fonte **comentado**:

```
/* Comentários são incluídos dentro dos marcadores a
vermelho ("barra-oblíqua asterisco" e "asterisco
barra-oblíqua") e podem ocupar múltiplas linhas */
// Em C99 podemos definir comentários de linha única
// utilizando duas barras oblíquas no início
#include <stdio.h> /* A diretiva #include indica que
 o conteúdo do ficheiro header stdio.h, da biblioteca
 normalizada do C, deve ser utilizado/incluído */

int main(void) // declaração simplificada da função main()
{
 printf ("Gosto de programar em C!\n");
 /* chamar a função printf(), cuja interface se
 encontra na biblioteca stdio.h, para escrever
 a mensagem na saída padrão (terminal). O "\n"
 representa a mudança de linha */
 return 0; /* sair da função, devolvendo zero para
 o sistema operativo */
}
```

➤ Para construir o programa executável a partir do exemplo anterior são necessários três passos, contudo podemos combiná-los num único comando (usando **gcc** ou **clang**). Se executados em separado, os passos são os seguintes:

1. Efetuar o **pré-processamento** das diretivas (comandos que começam com o prefixo **#**) incluídas no código, utilizando o pré-processador

*A informação de todos os ficheiros header referenciados é reunida num único ficheiro fonte.*

2. **Compilar** o ficheiro com código fonte, gerando um ficheiro objeto

*O nome dos ficheiros objeto geralmente tem a extensão **.o***

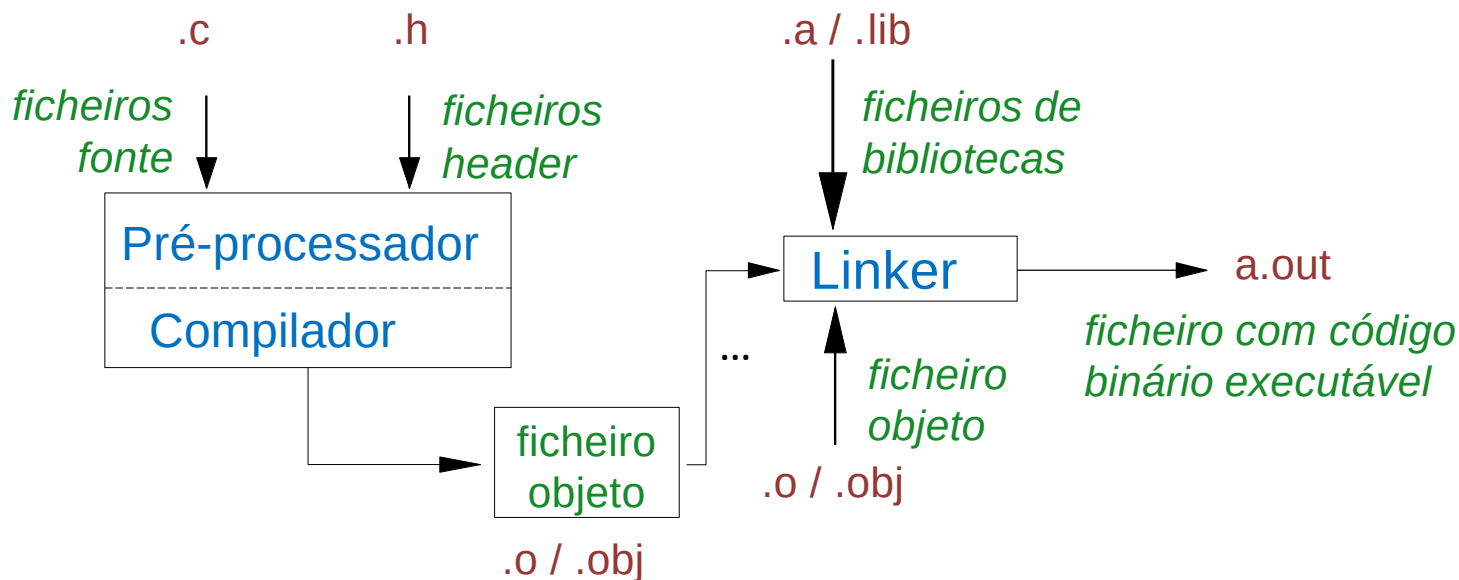
**gcc -c 01-programa.c -o 01-programa.o**

➤ *este comando efetua o pré-processamento e a compilação*

3. **Combinar/ligar** o ficheiro objeto gerado no passo anterior com o código objeto das bibliotecas necessárias, utilizando o **linker** (através do **gcc**)

**gcc 01-programa.o -o 01-programa**

- Editar os ficheiros com código fonte (extensão **.c** e **.h**)  
*Formato legível por humanos*
- Compilar os ficheiros fonte (**.c**) para ficheiros objeto (**.o** ou **.obj**)  
*Formato legível pela máquina*
- Ligar os ficheiros resultantes da compilação num ficheiro binário executável  
*Formato legível pela máquina*
- Executar e depurar o programa; se necessário alterar o código fonte



- **Pré-processador** prepara o código fonte para a compilação: junta os ficheiros *header*, substitui constantes e macros pelo seu valor, selecciona o código que vai ser compilado → ajuda a tornar o código mais legível e mais flexível para correr em diferentes ambientes

*o resultado são ficheiros fonte em formato texto*

- **Compilador** traduz o código fonte em código de mais baixo nível: máquina (ou *assembly*)

*código máquina (ou assembly) nativo da arquitetura alvo*

- **Linker** - Liga os ficheiros objeto num único programa executável

*Ao correr por cima do SO, o executável pode chamar funções de bibliotecas colocadas em memória dinamicamente e de bibliotecas do próprio SO*

- Como foi dito, as tarefas de **pré-processar**, **compilar** e **ligar** são normalmente efetuadas com um único comando e utilizando um único programa, desde que instruído com as opções certas

*Exemplos deste programa: clang e gcc*

- Nesta UC vamos usar como compilador de C o **gcc**:  
**gcc** - GNU Compiler Collection <https://gcc.gnu.org>
- O **gcc** é facilmente instalado em Linux e no MacOS:  
Por exemplo, em Ubuntu pode usar-se o comando:  
**sudo apt install build-essential gdb**
- Em Windows pode instalar-se o **gcc** utilizando o software **MinGW-x64**:  
<https://sourceforge.net/projects/mingw-w64/files/mingw-w64/mingw-w64-release/mingw-w64-v9.0.0.zip>  
Ou então, instalando o Windows Subsystem for Linux (WSL). Ajuda em:  
<https://docs.microsoft.com/en-us/windows/wsl/install-win10>
- Exemplo de comandos para compilar+montar e ligar:
  - Compilar e montar: **gcc -c main.c -o main.o**
  - Ligar: **gcc main.o -o main**

- As **funções** são a peça fundamental da **modularidade** permitida pela linguagem C

Um **programa modular** é composto por várias módulos/ficheiros fonte

- A **definição** **duma função** inclui:

- **cabeçalho** da função
- **corpo** da função

*A **definição** é a implementação da função*

- O **protótipo** (ou **declaração** ou **assinatura**) **duma função** é o cabeçalho dessa função e especifica como a função é chamada

*A **declaração** permite usar a função antes da sua definição, ou seja, permite compilar a código que use uma função sem conhecer a implementação dessa função. A implementação pode estar noutro ficheiro fonte ou numa biblioteca já compilada*

- A sintaxe **duma declaração** de função é a seguinte:

**tipo\_devolvido nome\_função (tipo\_arg1, tipo\_arg2, ...);**



- Definir uma **função dentro de outra** função não é permitido em C  
    ➤ **para nós módulo = ficheiro fonte/objeto**
- As funções são implicitamente declaradas como **extern**, ou seja, são visíveis para outros ficheiros
- Usando o especificador **static** antes da declaração da função, esta passa a ser visível apenas no ficheiro em que é definida
- Os **argumentos** duma função são **variáveis locais** inicializadas com os valores com que a função é chamada ... e guardados na pilha
- **C permite recursividade** – uma função que se chama a ela própria
- Os argumentos duma função não são obrigatórios → usa-se **void**  
`int func1(void);`
- O tipo do **valor devolvido** por uma função também pode ser **void**, quando a função não devolve nada explicitamente  
`void func2(int, float);`

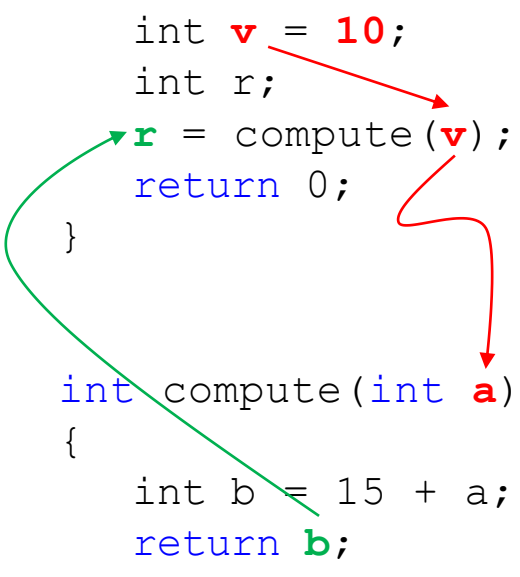
# Exemplo de Programa (ou Módulo)

```
#include <stdio.h> /* usar o ficheiro header */
#define NUMERO 5 /* constante */

int compute(int); /* protótipo da função compute */

int main(int argc, char *argv[]) /* função main */
{
 int v = 10; /* declaração duma variável */
 int r;
 r = compute(v); /* chamar a função compute */
 return 0; /* terminar a função main */
}

int compute(int a) /* definição da função compute */
{
 int b = 15 + a; /* corpo da função */
 return b; /* valor devolvido pela função */
}
```



# Posição de início dum programa: main()

- Cada programa executável deve conter pelo menos uma definição duma função e essa função tem que ser a `main()`
- A função `main()` é o **ponto de início** (da execução) dum programa
- A declaração da `main()` assume uma de duas formas básicas:

## 1. Formato sem argumentos:

```
int main(void)
{
 ...
}
```

## 2. Formato que permite receber argumentos do SO:

```
int main(int argc, char *argv[]) // ou `... char **argv'
{
 ...
}
```

# Argumentos da função `main()`

- Ao iniciar a execução dum programa, o SO passa ao programa o número de argumentos (`argc`) e os argumentos (`argv`)
  - O primeiro argumento é o nome do programa

```
int main(int argc, char *argv[])
{
 int v;
 v = 10;
 v = v + 1;
 return argc;
}
```

01-var.c

- O programa é **terminado** pelo `return` na função `main()`
- O valor devolvido é passado ao SO e pode ser utilizado, por exemplo, para saber como correu a execução do programa

- Ao construir o programa com o compilador **gcc**, se indicarmos nada em termos de paragem do processo, ele executa automaticamente as fases de compilação, montagem e ligação necessárias para obter o programa executável **a.out**

**gcc 01-var.c**

- O nome do ficheiro de saída pode ser especificado, neste exemplo escolhemos o nome **01-var**:

**gcc 01-var.c -o 01-var**

- Agora o programa pode ser executado com o comando:

**./01-var**

- A compilação e execução podem ser agrupadas num único comando:

**gcc 01-var.c -o 01-var ; ./01-var**

- A execução pode ser condicionada ao sucesso da compilação:

**gcc 01-var.c -o 01-var && ./01-var**

# Demo 01-var.c: executar o programa no terminal *bash*

- O **valor devolvido** pelo programa é guardado na **variável de ambiente \$?**
- Execução do programa com diferentes números de argumentos

```
./01-var
```

```
echo $?
```

```
1
```

```
./01-var ; echo $?
```

```
1
```

```
./01-var 1 2 3 ; echo $?
```

```
4
```

```
./01-var abcd ; echo $?
```

```
2
```

- Contextualização da linguagem C
- Características mais relevantes da linguagem C
- Estrutura dum programa em C
- Tipos de dados primários do C
- Programar em C
- **Valores e variáveis**
- Operadores e expressões
- Entrada e saída normalizadas
- C89 e C99

# Literais: escrever valores de um determinado tipo

- Um **valor** (ou **constante**) de um determinado tipo suportado pelo C é chamado um **literal**
- O C permite especificar **6 tipos** de literais:
  - Inteiro
  - Real
  - Caratere
  - Cadeia de carateres
  - Enumerado
  - Simbólico

`#define NUMERO 10`



- Os literais inteiros são guardados num dos tipos inteiros do C: `char` , `short`, `int`, `long` , `long long` e as variantes `signed` e `unsigned`
- Tipos de literais inteiros permitidos:

|               |         |                       |
|---------------|---------|-----------------------|
| Decimal       | 450932  |                       |
| Hexadecimal   | 0x12ADF | (começa com 0x ou 0X) |
| Octal         | 031567  | (começa com zero)     |
| unsigned      | 12345U  | (sufixo U ou u)       |
| long          | 12345L  | (sufixo L ou l)       |
| unsigned long | 12345ul | (sufixo UL ou ul)     |
| long long     | 12345LL | (sufixo LL ou ll)     |

- Sem sufixo, um literal inteiro é guardado como `int`

# Literais reais

- Os valores reais podem ser **escritos**:
  - com ponto flutuante: `23.1`      `-0.678`
  - em notação científica: `31.4e-3`      `-2.78E-5`
- Os literais reais são **guardados no formato de vírgula flutuante**, utilizando **a norma IEEE 754**
- **Tipos de dados reais** (formas de armazenamento) disponíveis em C:
  - **double** → armazenamento utilizado por omissão, quando o literal não explicitar outro tipo de armazenamento
  - **float** → quando o literal **termina com o sufixo F ou f**  
Exemplo: `float f = 17.6f;`
  - **long double** → quando o **literal termina com o sufixo L ou l**  
Exemplo: `long double ld = -25.74L;`

- Forma de os **escrever**: um caratere rodeado por **uma plica antes e uma plica depois**

`'a'`, `'B'`, `'\n'` (mudança de linha)

- Um caratere é **guardado** como um **inteiro**, e o valor corresponde ao seu **código ASCII**

`'0'` -> 48 , `'A'` -> 65 , `'a'` -> 97

# Literais do tipo cadeia de caracteres ↔ *strings*

- Forma de os **escrever**: uma sequência de caracteres e caracteres de controlo (sequências de *escape*) **entre duas plicas duplas**  
"Cadeia de caracteres com mudança linha no fim\n"
- Cadeias de caracteres separadas por espaços em branco são reunidas numa única cadeia. Por exemplo, estas 2 cadeias escritas desta forma:  
"Cadeia de caracteres" "com mudança linha no fim\n"  
são reunidas em:  
"Cadeia de caracteres com mudança linha no fim\n"
- Forma de os **armazenar**:
  - As cadeias de caracteres são guardadas num **vetor** do tipo **char**, acrescentando **no fim o caractere '\0'** (código ASCII zero)
  - Por exemplo, o literal "palavra" é armazenada como:

|     |     |     |     |     |     |     |      |
|-----|-----|-----|-----|-----|-----|-----|------|
| 'p' | 'a' | 'l' | 'a' | 'v' | 'r' | 'a' | '\0' |
|-----|-----|-----|-----|-----|-----|-----|------|

# Literais do Tipo Enumerado

- Um tipo enumerado é um **conjunto finito** de etiquetas (palavras), em que **a ordem** normalmente **não é relevante**
- Forma de os escrever: `enum {nome1, nome2, ..., nomeN}`
- Os literais enumerados são na realidade do tipo inteiro, em que cada etiqueta é codificada com um valor inteiro diferente
- Por omissão, a codificação das etiquetas começa em 0 para a primeira do conjunto, 1 para a segunda, 2 para a terceira, e assim sucessivamente

```
enum {ESPADAS, PAUS, COPAS, OUROS}
```

- Caso seja preciso, podemos especificar o código de cada etiqueta, utilizando a notação: **nome=valor**

```
enum {ESPADAS=10, PAUS, COPAS=15, OUROS=5} // PAUS=11
```

- Embora pouco comum, podemos percorrer as etiquetas dum tipo enumerado com um ciclo:

```
enum {ESPADAS=1, PAUS, COPAS, OUROS}
for (int i=ESPADAS; i<OUROS; ++i) { ... }
```

- Forma de os escrever: através de diretiva **#define** do pré-processador  
**#define NOME valor**
- O **valor** do literal pode ser uma **constante** ou uma **expressão** envolvendo variáveis, constantes e outros literais simbólicos
- Cada **#define** deve estar numa linha diferente  
**#define PI 3.14159**  
**#define AREA (10.0+PI\*raio\*raio)**
- Normalmente o **nome** do literal é escrito com **maiúsculas**
- Antes de se compilar o código, o **pré-processador faz a substituição dos nomes** das constantes definidas pelo seu **valor**
- Deve utilizar-se **parênteses curvos** a envolver a expressão dum literal simbólico para garantir uma **avaliação correta** da expressão
- **5.0\*AREA** será diferente consoante a expressão de **AREA** inclui parênteses externos ou não:  
**5.0\*(10.0+PI\*raio\*raio) ≠ 5.0\*10.0+PI\*raio\*raio**

# Variável com o modificador **const**

- Utilizando a palavra-chave **const** marcamos uma variável como tendo um **valor constante**
- Além do valor atribuído na definição da variável, não é permite atribuir-lhe outro valor
- Um valor constante pode ser definido da seguinte forma:

```
const float pi = 3.14159;
```

- Em contraste com a constante simbólica

```
#define PI 3.14159265
```

as variáveis constantes (**pi**, neste caso) têm tipo, permitindo ao compilador **verificar** a sua correta utilização ⇔ *type checking*

# Exemplo: soma de dois valores

```
#include <stdio.h>

int main(void)
{
 int sum; // definição de variável local do tipo int
 sum = 100 + 43; /* definir a expressão que dará
 o valor de sum */
 printf("A soma de 100 com 43 e' %i\n", sum);
 /* a formatação %i indica ao printf que deve
 escrever sum como sendo um inteiro */
 return 0;
}
```

- A variável **sum**, do tipo **int**, representa um número inteiro
- O valor de **sum** é **guardado em memória**
- **sum** é assim um **nome simbólico** que representa a posição de memória onde o valor inteiro é guardado



# Exemplo: soma de duas variáveis

```
#include <stdio.h>

int main(void)
{
 int var1; // declaração/definição de variável
 int var2 = 10; // definição com inicialização de variável
 int sum;

 var1 = 13;

 sum = var1 + var2;

 printf("A soma de %i com %i é' %i\n", var1, var2, sum);

 return 0;
}
```

- As variáveis **var1**, **var2** e **sum** representam três posições diferentes na memória, reservadas automaticamente, onde se guardam três valores inteiros

- A **declaração duma variável** tem a seguinte forma:  
**especificadores-de-declaracao declaradores;**
- Os especificadores de declaração são:
  - **Tipo de armazenamento:** no máximo um dos tipos **auto**, **static**, **extern**, ou **register**
  - **Quantificadores de tipo:** **const**, **volatile**, **restrict**  
*São permitidos 0+ quantificadores de tipo*
  - **Especificadores de tipo:** **void**, **char**, **short**, **int**, **long**, **float**, **double**, **signed**, **unsigned**

Além destes tipos, também podem ser utilizados os especificadores do tipo **struct** e **union**

Finalmente, também podem ser utilizados tipos definidos pelo utilizador com **typedef**

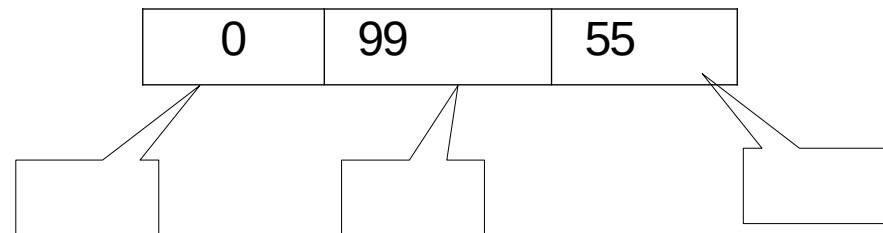
## variáveis do tipo **unsigned char**

```
unsigned char var1;
unsigned char var2;
unsigned char sum;
```

```
var1 = 13;
var2 = 10;
```

```
sum = var1 + var2;
```

- Para cada variável é reservado 1 byte
- O conteúdo da memória não é definido após a reserva do espaço
- O nome duma variável é uma forma do programador ter acesso à respetiva posição de memória
- O valor da variável é o conteúdo da posição de memória



## variáveis do tipo **unsigned char**

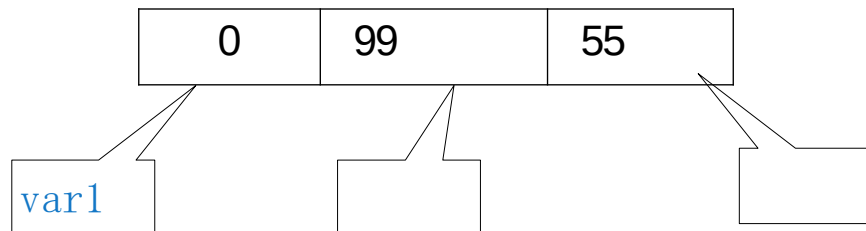
```
unsigned char var1;
unsigned char var2;
unsigned char sum;
```

```
var1 = 13;
```

```
var2 = 10;
```

```
sum = var1 + var2;
```

- Para cada variável é reservado 1 byte
- O conteúdo da memória não é definido após a reserva do espaço
- O nome duma variável é uma forma do programador ter acesso à respetiva posição de memória
- O valor da variável é o conteúdo da posição de memória



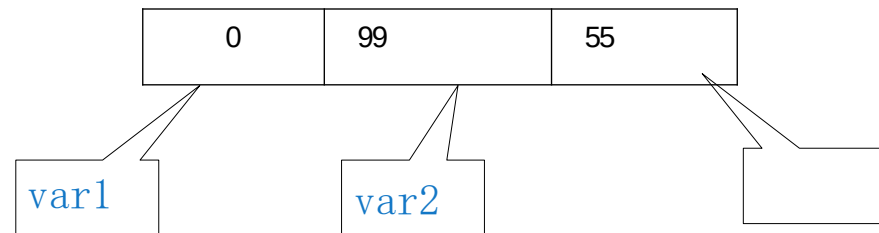
## variáveis do tipo **unsigned char**

```
unsigned char var1;
unsigned char var2;
unsigned char sum;
```

```
var1 = 13;
var2 = 10;
```

```
sum = var1 + var2;
```

- Para cada variável é reservado 1 byte
- O conteúdo da memória não é definido após a reserva do espaço
- O nome duma variável é uma forma do programador ter acesso à respetiva posição de memória
- O valor da variável é o conteúdo da posição de memória



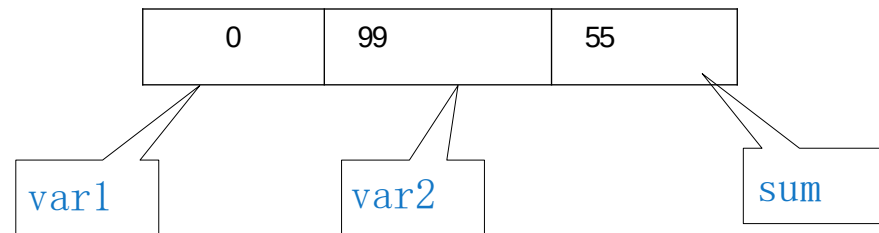
## variáveis do tipo **unsigned char**

```
unsigned char var1;
unsigned char var2;
unsigned char sum;
```

```
var1 = 13;
var2 = 10;
```

```
sum = var1 + var2;
```

- Para cada variável é reservado 1 byte
- O conteúdo da memória não é definido após a reserva do espaço
- O nome duma variável é uma forma do programador ter acesso à respetiva posição de memória
- O valor da variável é o conteúdo da posição de memória



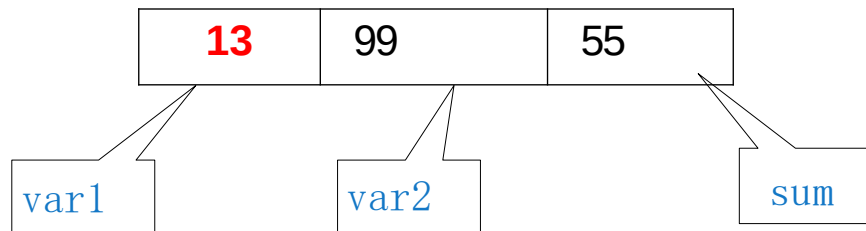
## variáveis do tipo **unsigned char**

```
unsigned char var1;
unsigned char var2;
unsigned char sum;
```

```
var1 = 13;
var2 = 10;
```

```
sum = var1 + var2;
```

- Para cada variável é reservado 1 byte
- O conteúdo da memória não é definido após a reserva do espaço
- O nome duma variável é uma forma do programador ter acesso à respetiva posição de memória
- O valor da variável é o conteúdo da posição de memória



## variáveis do tipo **unsigned char**

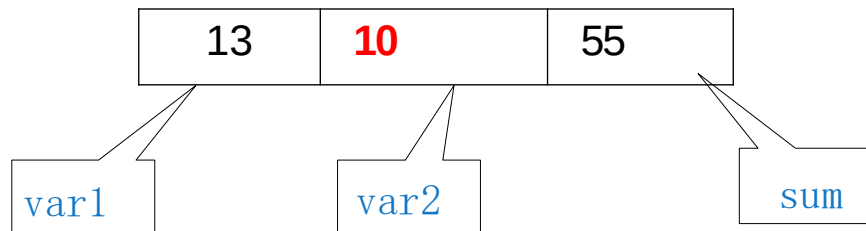
```
unsigned char var1;
unsigned char var2;
unsigned char sum;
```

```
var1 = 13;
```

```
var2 = 10;
```

```
sum = var1 + var2;
```

- Para cada variável é reservado 1 byte
- O conteúdo da memória não é definido após a reserva do espaço
- O nome duma variável é uma forma do programador ter acesso à respetiva posição de memória
- O valor da variável é o conteúdo da posição de memória





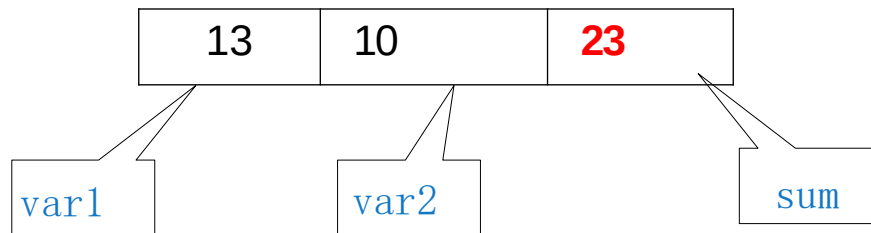
## variáveis do tipo **unsigned char**

```
unsigned char var1;
unsigned char var2;
unsigned char sum;
```

```
var1 = 13;
var2 = 10;
```

```
sum = var1 + var2;
```

- Para cada variável é reservado 1 byte
- O conteúdo da memória não é definido após a reserva do espaço
- O nome duma variável é uma forma do programador ter acesso à respetiva posição de memória
- O valor da variável é o conteúdo da posição de memória



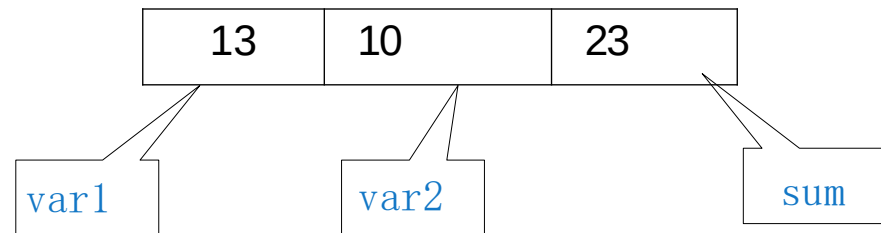
## variáveis do tipo **unsigned char**

```
unsigned char var1;
unsigned char var2;
unsigned char sum;
```

```
var1 = 13;
var2 = 10;
```

```
sum = var1 + var2;
```

- Para cada variável é reservado 1 byte
- O conteúdo da memória não é definido após a reserva do espaço
- O nome duma variável é uma forma do programador ter acesso à respetiva posição de memória
- O valor da variável é o conteúdo da posição de memória



# Atribuição, variáveis e memória: variáveis **int**

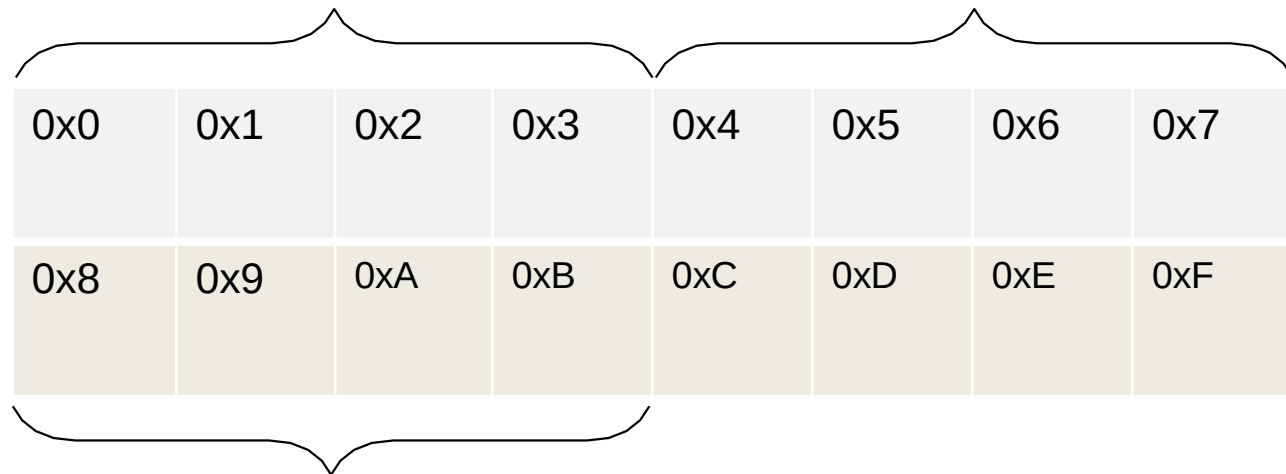
```
int var1;
int var2;
int sum;
```

```
// 00 00 00 13
var1 = 13;
```

```
// x00 x00 x01 xF4
var2 = 500;
```

```
sum = var1 + var2;
```

- Para uma variável do tipo **int** são reservados 4 bytes
- O tamanho pode ser obtido com a função **sizeof(int)**
- O conteúdo da memória não é definido no momento da definição das 3 variáveis



500 (decimal) é 0x01F4 (hex)

513 (decimal) é 0x0201 (hex)

*Para as arquiteturas Intel x86 e x86-64, os valores (dos tipos que ocupam múltiplos bytes) são armazenados na ordenação **little-endian***

# Atribuição, variáveis e memória: variáveis **int**

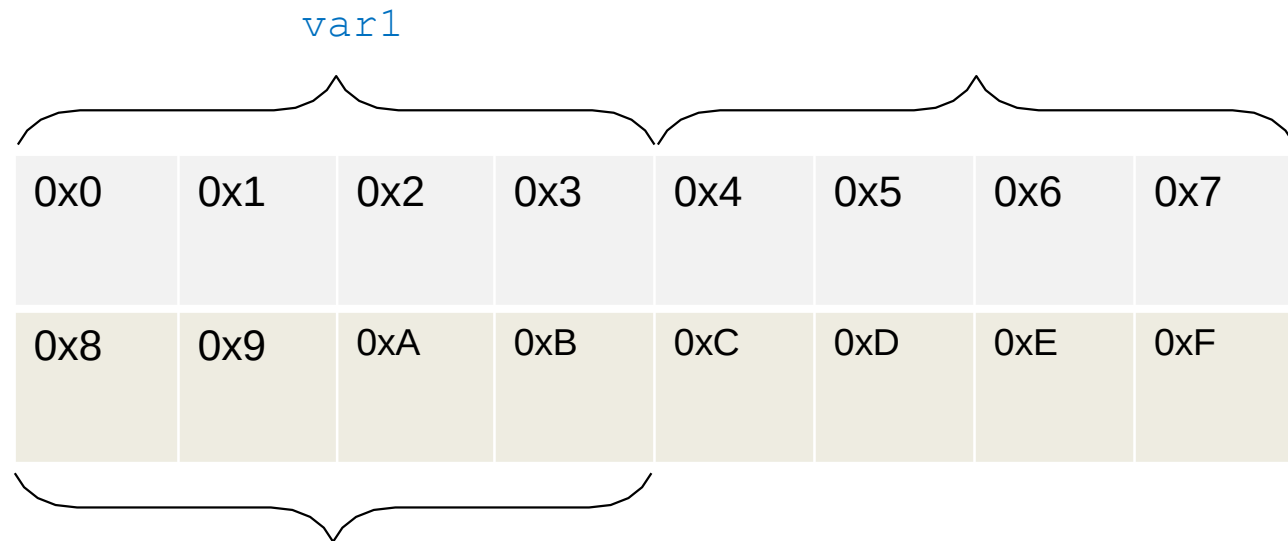
```
int var1;
int var2;
int sum;
```

```
// 00 00 00 13
var1 = 13;
```

```
// x00 x00 x01 xF4
var2 = 500;
```

```
sum = var1 + var2;
```

- Para uma variável do tipo **int** são reservados 4 bytes
- O tamanho pode ser obtido com a função **sizeof(int)**
- O conteúdo da memória não é definido no momento da definição das 3 variáveis



500 (decimal) é 0x01F4 (hex)

513 (decimal) é 0x0201 (hex)

*Para as arquiteturas Intel x86 e x86-64, os valores (dos tipos que ocupam múltiplos bytes) são armazenados na ordenação **little-endian***

# Atribuição, variáveis e memória: variáveis **int**

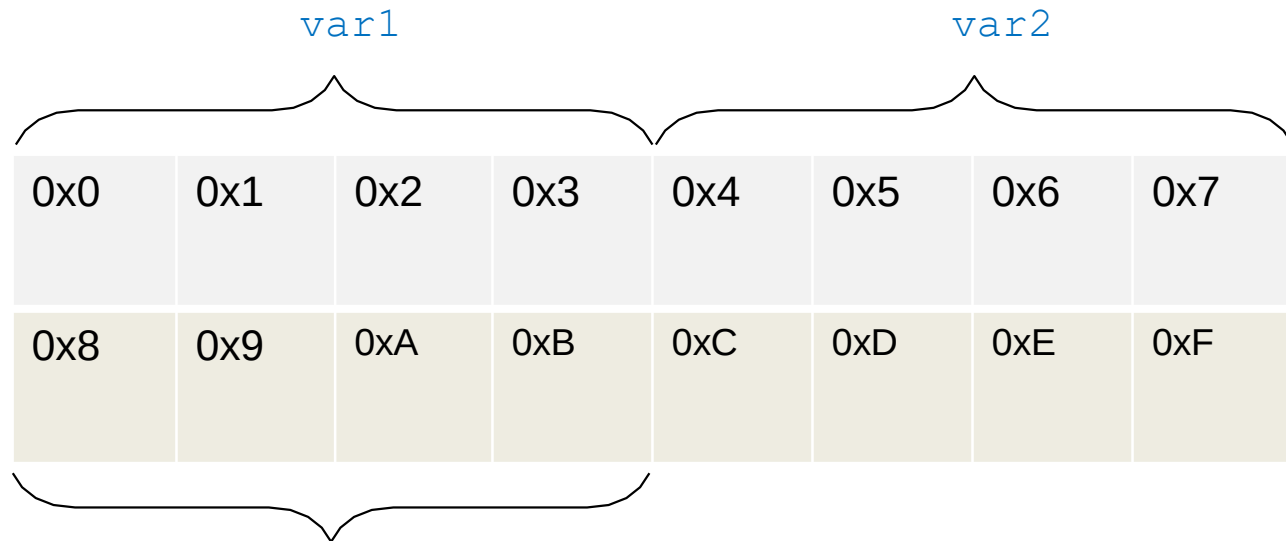
```
int var1;
int var2;
int sum;
```

```
// 00 00 00 13
var1 = 13;
```

```
// x00 x00 x01 xF4
var2 = 500;
```

```
sum = var1 + var2;
```

- Para uma variável do tipo **int** são reservados 4 bytes
- O tamanho pode ser obtido com a função **sizeof(int)**
- O conteúdo da memória não é definido no momento da definição das 3 variáveis



500 (decimal) é 0x01F4 (hex)

513 (decimal) é 0x0201 (hex)

*Para as arquiteturas Intel x86 e x86-64, os valores (dos tipos que ocupam múltiplos bytes) são armazenados na ordenação **little-endian***

# Atribuição, variáveis e memória: variáveis **int**

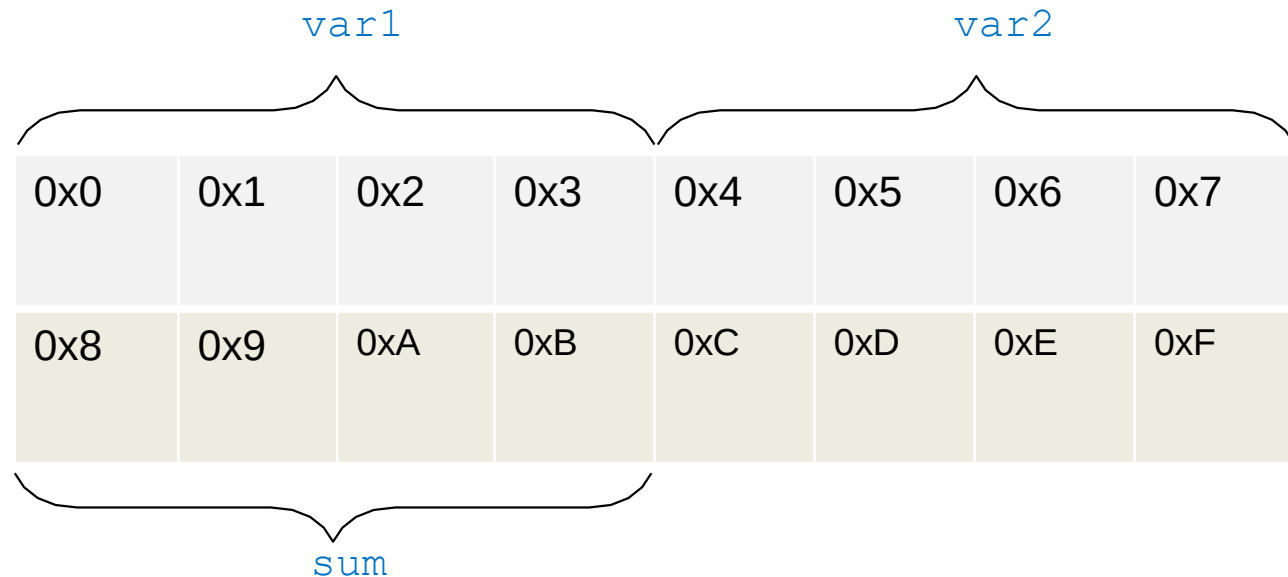
```
int var1;
int var2;
int sum;
```

```
// 00 00 00 13
var1 = 13;
```

```
// x00 x00 x01 xF4
var2 = 500;
```

```
sum = var1 + var2;
```

- Para uma variável do tipo **int** são reservados 4 bytes
- O tamanho pode ser obtido com a função **sizeof(int)**
- O conteúdo da memória não é definido no momento da definição das 3 variáveis



500 (decimal) é 0x01F4 (hex)  
513 (decimal) é 0x0201 (hex)

*Para as arquiteturas Intel x86 e x86-64, os valores (dos tipos que ocupam múltiplos bytes) são armazenados na ordenação **little-endian***

# Atribuição, variáveis e memória: variáveis **int**

```
int var1;
int var2;
int sum;
```

```
// 00 00 00 13
```

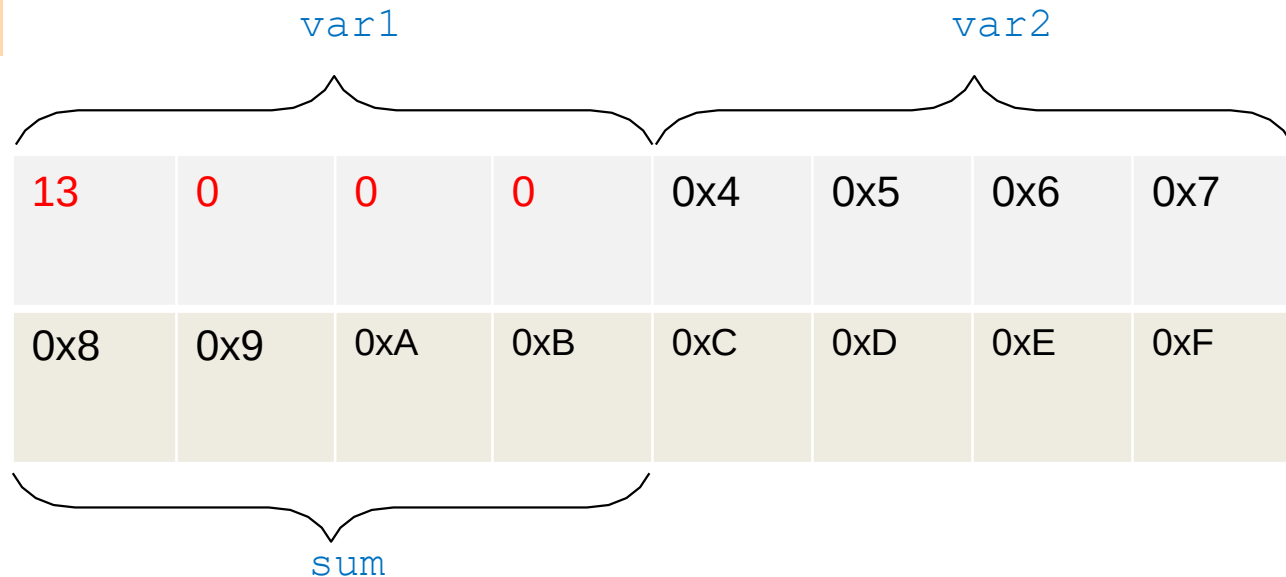
```
var1 = 13;
```

```
// x00 x00 x01 xF4
```

```
var2 = 500;
```

```
sum = var1 + var2;
```

- Para uma variável do tipo **int** são reservados 4 bytes
- O tamanho pode ser obtido com a função **sizeof(int)**
- O conteúdo da memória não é definido no momento da definição das 3 variáveis



500 (decimal) é 0x01F4 (hex)

513 (decimal) é 0x0201 (hex)

*Para as arquiteturas Intel x86 e x86-64, os valores (dos tipos que ocupam múltiplos bytes) são armazenados na ordenação **little-endian***

# Atribuição, variáveis e memória: variáveis **int**

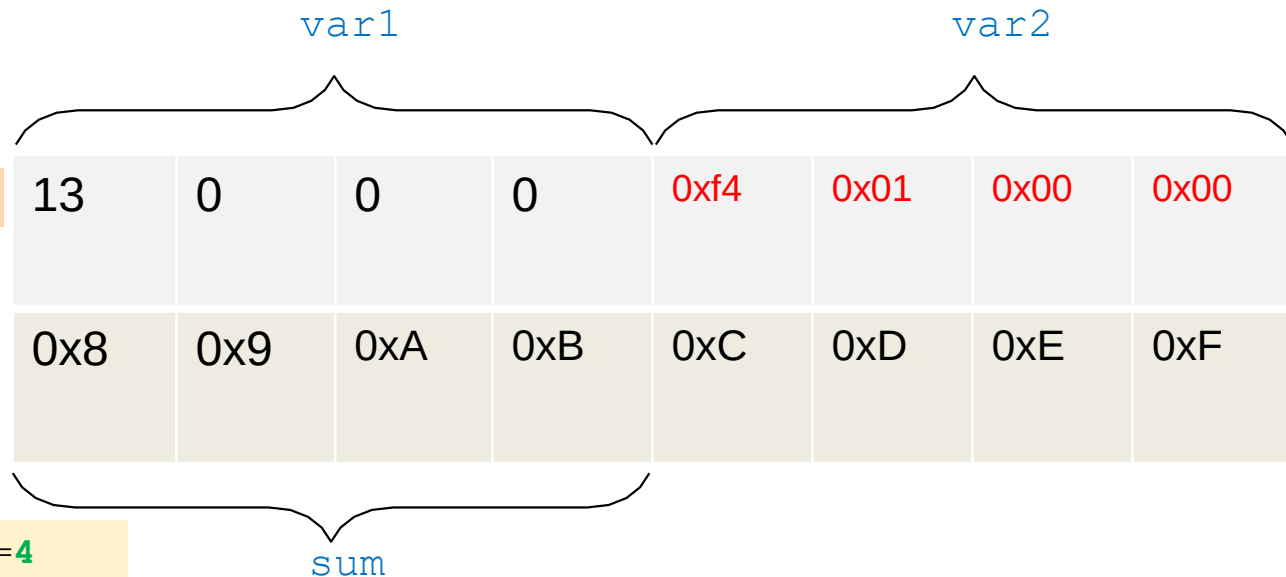
```
int var1;
int var2;
int sum;
```

```
// 00 00 00 13
var1 = 13;
```

```
// x00 x00 x01 xF4
var2 = 500;
```

```
sum = var1 + var2;
```

- Para uma variável do tipo **int** são reservados 4 bytes
- O tamanho pode ser obtido com a função **sizeof(int)**
- O conteúdo da memória não é definido no momento da definição das 3 variáveis



```
500/16 -> Quoc=31 resto=4
31/16 -> Quoc=1 resto=15=F
1/16 -> Quoc=0 resto=1
```

500 (decimal) é 0x01F4 (hex)  
513 (decimal) é 0x0201 (hex)

*Para as arquiteturas Intel x86 e x86-64, os valores (dos tipos que ocupam múltiplos bytes) são armazenados na ordenação **little-endian***



# Atribuição, variáveis e memória: variáveis **int**

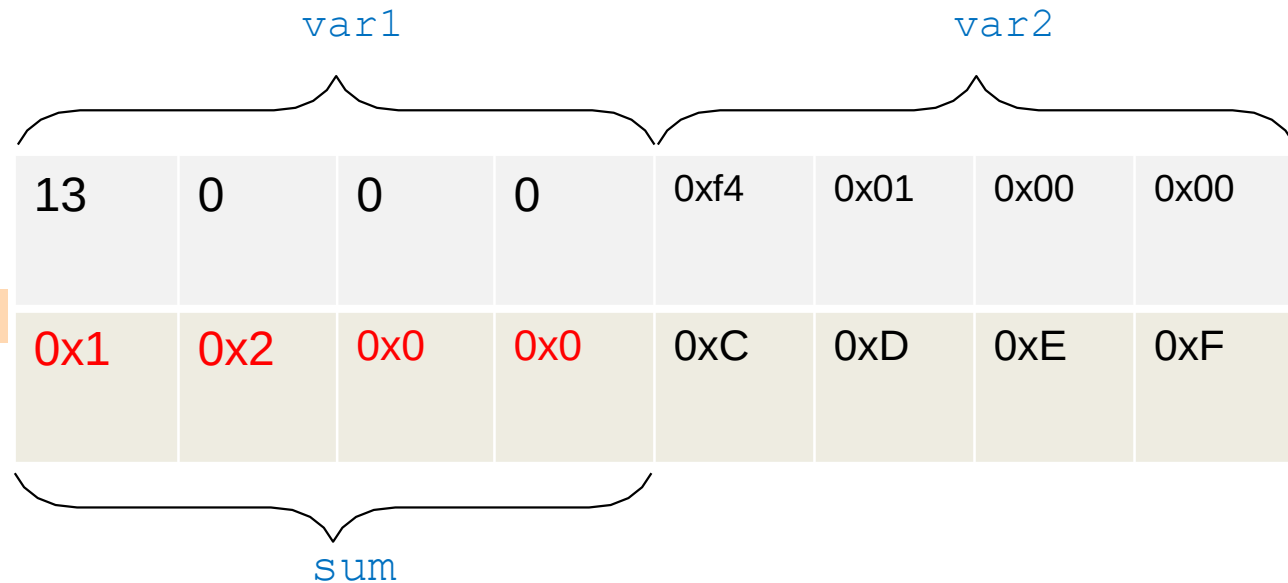
```
int var1;
int var2;
int sum;
```

```
// 00 00 00 13
var1 = 13;
```

```
// x00 x00 x01 xF4
var2 = 500;
```

```
sum = var1 + var2;
```

- Para uma variável do tipo **int** são reservados 4 bytes
- O tamanho pode ser obtido com a função **sizeof(int)**
- O conteúdo da memória não é definido no momento da definição das 3 variáveis



500 (decimal) é 0x01F4 (hex)

513 (decimal) é 0x0201 (hex)

*Para as arquiteturas Intel x86 e x86-64, os valores (dos tipos que ocupam múltiplos bytes) são armazenados na ordenação **little-endian***

- Contextualização da linguagem C
- Características mais relevantes da linguagem C
- Estrutura dum programa em C
- Tipos de dados primários do C
- Programar em C
- Valores e variáveis
- **Operadores e expressões**
- Entrada e saída normalizadas
- C89 e C99

- Uma **expressão** descreve **como calcular um valor** a partir das entradas utilizadas
- Uma expressão é composta por **operandos**, **operadores** e **parênteses**

- literal
- variável
- constante
- chamada de função

- operador unário
- operador binário

- A **ordem de avaliação** das operações é determinada pela **precedência** (prioridade) do operador e pela **associatividade**

- Exemplo:

$10 + x * y$  // ordem de avaliação  $10 + (x * y)$

$10 + x + y$  // ordem de avaliação  $(10 + x) + y$

*\* tem maior prioridade do que +  
+ é associativo da esquerda para a direita*

# Operadores

- Os operadores são caracteres selecionados (ou uma sequência de caracteres) que a linguagem C destina à escrita de expressões
- Podemos distinguir cinco tipos de **operadores binários**:
  - Operadores **aritméticos** → adição, subtração, multiplicação, divisão
  - Operadores **relacionais** → comparação de valores (menor que, maior que, ...)
  - Operadores **lógicos** → **E** lógico e **OU** lógico
  - Operadores **ao nível do bit** → **E**, **OU** e **OU-exclusivo** bit-a-bit, deslocamento à esquerda e à direita
  - Operador de **atribuição** **=** → a variável de destino fica à esquerda do **=**
- Operadores **unários**:
  - Indicação de valor **positivo/negativo** → **+** e **-**  
*O operador - modifica o sinal da expressão*
  - **Incrementar/decrementar** uma variável da unidade → **++** e **--**
  - **Negação** lógica → **!**
  - **Negação** ao nível do bit → **~**
- Operador **ternário** **?:** → para definir uma expressão condicional

- **Atribuição** corresponde a definir o valor da variável, ou seja, corresponde a guardar o valor na posição da memória referenciada pelo nome da variável
- Operador de atribuição =  
 $\langle \text{E-valor} \rangle = \langle \text{expressão} \rangle$ 
  - O **lado esquerdo** do = tem que ser **uma posição de memória onde o valor da expressão pode ser guardado**
  - A **expressão** é um literal, uma variável, a chamada duma função,...
  - A atribuição é uma expressão e podemos usá-la em qualquer lugar onde seja permitido usar a expressão do tipo particular
  - Uma **instrução de atribuição** é uma declaração contendo o operador de atribuição = e terminada por ;

# Expressões aritméticas simples

- Para os tipos **numéricos** `int`, `short`, `char`, `float` e `double`, o C dispõe dos seguintes operadores:
  - operador unário de **mudança de sinal**: `-`
  - **adição** binária `+` e **subtração** binária `-`
  - **multiplicação** binária `*` e **divisão** binária `/`
- Para operações sobre **inteiros** existe ainda:
  - **módulo** binário (ou **resto** da divisão inteira) `%`
- Se os operandos são do mesmo tipo, o resultado da operação aritmética é naturalmente desse mesmo tipo
- Ao combinar numa operação tipos de dados `int` e `double`, o `int` é convertido para `double` e o resultados é do tipo `double`

*Conversão de tipo implícita*

# Operadores aritméticos: exemplo <sup>1/2</sup>

```
int a = 10;
int b = 3;
int c = 4;
int d = 5;
int resultado;
```

```
resultado = a - b; // subtração
printf("a - b = %i\n", resultado);
```

```
resultado = a * b; // multiplicação
printf("a * b = %i\n", resultado);
```

```
resultado = a / b; // divisão inteira
printf("a / b = %i\n", resultado);
```

```
resultado = a + b * c; // perceber a prioridade dos operadores
printf("a + b * c = %i\n", resultado);
```

```
printf("a * b + c * d = %i\n", a * b + c * d); // -> 50
printf("(a * b) + (c * d) = %i\n", (a * b) + (c * d)); // -> 50
printf("a * (b + c) * d = %i\n", a * (b + c) * d); // -> 350
```

```
// 01-ops-aritmeticos.c
```

# Operadores aritméticos: exemplo 2/2

```
#include <stdio.h>

int main(void)
{
 int x1 = 1;
 double y1 = 2.2357;
 float x2 = 2.5343f;
 double y2 = 2;

 printf("P1 = (%i, %f)\n", x1, y1);
 printf("P1 = (%i, %i)\n", x1, (int)y1);
 printf("P1 = (%f, %f)\n", (double)x1, (double)y1);
 printf("P1 = (%.3f, %.3f)\n", (double)x1, (double)y1);

 printf("P2 = (%f, %f)\n", x2, y2);

 double dx = (x1 - x2); // conversão implícita para float
 double dy = (y1 - y2); // e depois para double

 printf("(P1 - P2)=(%.3f, %0.3f)\n", dx, dy);
 printf("|P1 - P2|^2=%.2f\n", dx * dx + dy * dy);
 return 0;
}
```

// 01-pontos2d.c



- Contextualização da linguagem C
- Caraterísticas mais relevantes da linguagem C
- Estrutura dum programa em C
- Tipos de dados primários do C
- Programar em C
- Valores e variáveis
- Operadores e expressões
- **Entrada e saída normalizadas**
- C89 e C99

- Quando um programa é executado numa máquina com Sistema Operativo (SO) tem acesso a uma entrada de informação (**stdin**) e a uma saída de informação (**stdout**) normalizadas

*Programas que correm em microcontroladores (sem SO) não têm stdin nem stdout*

- Os canais **stdin** e **stdout** podem ser utilizados para comunicar com o utilizador
- A função mais elementar para entrada de texto é o **getchar()** e para saída de texto é o **putchar()**

*Ambas as funções são definidas na biblioteca normalizada do C e são acessíveis via **stdio.h***

- Para ler valores numéricos do utilizador, pode utilizar-se a função **scanf()**
- A função **printf()** fornece uma saída formatada, por exemplo, para escrever um número real com 4 casas decimais no terminal

- Definição da função em `stdio.h`:
- ```
int getchar(void) ;
```

```
int c;  
  
c = getchar() ;  
if (c == EOF)  
    ...
```

Notas:

- **EOF** é uma **constante** definida em **stdio.h**
- Se declararmos **c** do tipo **char**, em vez de **int** como fizemos, e depois compararmos com **EOF** o teste falha

- Definição da função em `stdio.h`:

```
int putchar(int c);
```

```
char c = 'x';
```

```
int b;
```

```
...
```

```
b = putchar((int) c);
```

```
if (b == EOF)
```

```
...
```

Exemplo: fazer o eco do caractere lido (1)

```
#include <stdio.h>

int main (void)
{
    int c;
    c = getchar();
    while (c != '\n') {
        putchar(c);
        c = getchar();
    }
    putchar('\n');

    return 0;
} // 01-eco-carattere.c
```

Exemplo: fazer o eco do caratere lido (2)

- Entrada do teclado vs. entrada de um ficheiro
 - Utilize um editor para escrever o texto “**ler caracteres de ficheiro**”, a usar como entrada do programa, num ficheiro com nome **input-eco.txt**
 - Execute o programa **01-eco-caratere** com a entrada redirecionada do ficheiro criado:
./01-eco-caratere < input-eco.txt
 - Repita a execução do programa **01-eco-caratere** e depois escreva no teclado “**ler caracteres do teclado**”:
./01-eco-caratere

A função `printf()`

- Como o `putchar()` apenas escreve um caractere de cada vez, acaba por ser pouco utilizada
- `printf()` é uma função da biblioteca normalizada bem mais adequada para escrever de forma formatada, uma vez que consegue converter vários tipos de entrada para um formato legível
- `int printf(const char* formato, ...)`
 - permite um número variável de argumentos após a *string* de formatação
 - a formatação especifica como é que cada um dos argumentos de entrada deve ser convertido (formatado) ao ser enviado para a saída

A função `printf()` : formatação (1)

- Cada especificador de formato do **`printf`** segue o formato:

`%[opcoes][largura][.precisao][tamanho]especificador`

onde:

`especificador` → é a parte mais importante e a única obrigatória,
e indica qual o **tipo de dados** a escrever

(consultar tabela do manual com as alternativas)

- ✓ Exemplo: **`"%f"`** para escrever um **`float`**

`largura` → a **quantidade de caracteres** a utilizar na escrita de um
argumento, preenchendo com espaços em branco aqueles que
não forem precisos para escrever esse argumento

- ✓ Exemplo: **`"%15f"`** escreve um **`float`** com 15 caracteres; se por
ex., o **`float`** apenas ocupar 7 caracteres o **`printf`** escreve 8
espaços em branco antes do **`float`**

A função `printf()` : formatação (2)

.precisao → quantos **dígitos à direita da casa decimal** são utilizados para escrever um número real

✓ Exemplos:

- `printf("%15.3f", 10.0*pi)` escreve o resultado como "**31.416**", com **três dígitos à direita** da casa decimal
- `printf("%15.3e", 10.0*pi)` escreve o resultado como "**3.142e+01**", com **três dígitos de precisão**

A função `printf()` : formatação (3)

opcoes → são campos opcionais e podem ser:

- Um sinal de menos (-) para alinhar o valor escrito à esquerda
- Um sinal de mais (+) para forçar a escrita do sinal do número
- Um **espaço em branco** para inibir a escrita do sinal do número
- Dígitos zero (0) para escrever zeros à esquerda em vez de espaços, quando o valor a escrever requer menos caracteres do que a largura indicada.
- Caractere cardinal (#) serve diferentes propósitos

A função `printf()` : formatação (4)

opcoes (continuação)

- Caratere cardinal (**#**) serve diferentes propósitos:
 - Se o valor a escrever for em octal ou hexadecimal, especifica que vai ser colocado um **0** ou **0x** no início do número (se ele for diferente de zero)
 - Se o valor a escrever for um **float/double**, o **#** obriga a escrever o ponto/vírgula decimal, mesmo que não haja dígitos à direita desse ponto

tamanho → modifica a largura a utilizar na escrita dos inteiros
(consultar tabelas para os detalhes)

A função `printf()` : manual da formatação (1)

`%[flags][width][.precision][length]specifier`

<i>specifier</i>	Output	Example
<code>d</code> or <code>i</code>	Signed decimal integer	392
<code>u</code>	Unsigned decimal integer	7235
<code>o</code>	Unsigned octal	610
<code>x</code>	Unsigned hexadecimal integer	7fa
<code>X</code>	Unsigned hexadecimal integer (uppercase)	7FA
<code>f</code>	Decimal floating point, lowercase	392.65
<code>F</code>	Decimal floating point, uppercase	392.65
<code>e</code>	Scientific notation (mantissa/exponent), lowercase	3.9265e+2
<code>E</code>	Scientific notation (mantissa/exponent), uppercase	3.9265E+2
<code>g</code>	Use the shortest representation: <code>%e</code> or <code>%f</code>	392.65
<code>G</code>	Use the shortest representation: <code>%E</code> or <code>%F</code>	392.65
<code>a</code>	Hexadecimal floating point, lowercase	-0xc.90fep-2
<code>A</code>	Hexadecimal floating point, uppercase	-0XC.90FEP-2
<code>c</code>	Character	a
<code>s</code>	String of characters	sample
<code>p</code>	Pointer address	b8000000
<code>n</code>	Nothing printed. The corresponding argument must be a pointer to a signed int. The number of characters written so far is stored in the pointed location.	
<code>%</code>	A <code>%</code> followed by another <code>%</code> character will write a single <code>%</code> to the stream.	%

`%[flags][width][.precision][length]specifier`

<i>flags</i>	description
-	Left-justify within the given field width; Right justification is the default (see <i>width</i> sub-specifier).
+	Forces to precede the result with a plus or minus sign (+ or -) even for positive numbers. By default, only negative numbers are preceded with a - sign.
(space)	If no sign is going to be written, a blank space is inserted before the value.
#	Used with o, x or X specifiers the value is preceeded with 0, 0x or 0X respectively for values different than zero. Used with a, A, e, E, f, F, g or G it forces the written output to contain a decimal point even if no more digits follow. By default, if no digits follow, no decimal point is written.
0	Left-pads the number with zeroes (0) instead of spaces when padding is specified (see <i>width</i> sub-specifier).

<i>width</i>	description
(number)	Minimum number of characters to be printed. If the value to be printed is shorter than this number, the result is padded with blank spaces. The value is not truncated even if the result is larger.
*	The <i>width</i> is not specified in the <i>format</i> string, but as an additional integer value argument preceding the argument that has to be formatted.

%[flags][width][.precision][length]specifier

.precision	description
.number	For integer specifiers (d, i, o, u, x, X): <i>precision</i> specifies the minimum number of digits to be written. If the value to be written is shorter than this number, the result is padded with leading zeros. The value is not truncated even if the result is longer. A <i>precision</i> of 0 means that no character is written for the value 0. For a, A, e, E, f and F specifiers: this is the number of digits to be printed after the decimal point (by default, this is 6). For g and G specifiers: This is the maximum number of significant digits to be printed. For s: this is the maximum number of characters to be printed. By default all characters are printed until the ending null character is encountered. If the period is specified without an explicit value for <i>precision</i>, 0 is assumed.
.*	The <i>precision</i> is not specified in the <i>format</i> string, but as an additional integer value argument preceding the argument that has to be formatted.

A função `printf()` : manual da formatação (4)



Universidade do Minho
Escola de Engenharia

%[flags][width][.precision][length]specifier

	specifiers						
length	d i	u o x X	f F e E g G a A	c	s	p	n
<i>(none)</i>	int	unsigned int	double	int	char*	void*	int*
hh	signed char	unsigned char					signed char*
h	short int	unsigned short int					short int*
l	long int	unsigned long int		wint_t	wchar_t*		long int*
ll	long long int	unsigned long long int					long long int*
j	intmax_t	uintmax_t					intmax_t*
z	size_t	size_t					size_t*
t	ptrdiff_t	ptrdiff_t					ptrdiff_t*
L			long double				

- **Escrever** no terminal o número **33.3** (*double*):

<i>String</i> de formatação do <code>printf()</code>	Saída do <code>printf()</code>
<code>%.20f</code>	33.299999999999999999700000
<code>%14.10f</code>	33.3000000000
<code>%7.1f</code>	33.3

01-printf333.c

A função `printf()` : tipos de conversão

Tipo de dados a escrever	Formatação do print
<code>char</code>	<code>%c</code>
<code>unsigned int</code>	<code>%u</code> (em decimal) <code>%o</code> (em octal) <code>%x, %X</code> (em hex) (<code>%lu, %lo, %lx</code> no caso <code>long int</code>)
<code>signed int</code>	<code>%d, %i</code> (em decimal) (<code>%ld, %li</code> para o caso <code>long int</code>)
<code>float / double</code>	<code>%f</code>
<code>float / double</code>	<code>%e, %E</code> (notação científica)
Cadeia de caracteres	<code>%s</code>

A função `printf()` : exemplo

➤ Programa:

```
char  c = 'a';                                // 01-printf2.c
int    i = 9999;
float  f = 3.1415926535897932;

printf("c = %c (%o em octal)\n", c, c);

printf("i = %6d (%x em hexadecimal)\n", i, i);

printf("f = %8.5f (%e em notacao cientifica)\n", f, f);
```

Saída:

```
c = a (141 em octal)
i =   9999 (270f em hexadecimal) <- 2 espaços antes de 9999
f =  3.14159 (3.141593e+000 em notacao cientifica)
```

Lembrete: conversão octal/hexadecimal para decimal

➤ Base 16 ou hexadecimal:

$$2F3 = 2 * 16^2 + 15 * 16^1 + 3 = 755$$

➤ Base 8 ou octal:

$$463 = 4 * 8^2 + 6 * 8^1 + 3 = 307$$

Programa com saída para stdout: exemplo (1)

- Em vez de `printf()` podemos usar `fprintf()` explicitando que a saída é `stdout`, ou em alternativa `stderr`
- Ambas as funções estão declaradas em `stdio.h`

```
#include <stdio.h>

int main(int argc, char **argv)
{
    fprintf(stdout, "O meu primeiro programa em C!\n");
    fprintf(stdout, "O nome do ficheiro e\' \' \"%s\""\n", argv[0]);
    fprintf(stdout, "Foi executado com %d argumentos\n", argc);
    if (argc > 1) {
        fprintf(stdout, "Os argumentos sao:\n");
        for (int i = 1; i < argc; ++i) {
            fprintf(stdout, "  Arg %d e\' \' \"%s\""\n", i, argv[i]);
        }
    }
}
```

// 01-print-args.c

Programa com saída para stdout: exemplo (2)

- Ao incluirmos o ficheiro *header* `stdio.h` também são incluídos outros ficheiros *header* → por exemplo, aqueles que sejam incluídos em `stdio.h`

```
gcc 01-print-args.c -o 01-print-args  
./01-print-args primeiro segundo
```

```
O meu primeiro programa em C!  
O nome do ficheiro e' ". /01-print_args"  
Foi executado com 3 argumentos  
Os argumentos sao:  
  Arg 1 e' "primeiro"  
  Arg 2 e' "segundo"
```

Entrada formatada: **scanf** () (1)

- **scanf()** permite ler valores da entrada normalizada de acordo com a formatação fornecida

```
int scanf ( const char * formato, ... );
```

- O **1º argumento** da função é a *string* de formatação

*A sintaxe da formatação é similar à do **printf()***

- Um **espaço em branco** na *string* de formatação implica que o **scanf** lê e ignore todos os espaços em branco que encontrar antes do próximo caractere que não seja um espaço em branco
- **Cada argumento além do 1º** é a posição memória correspondente a uma variável onde se vai guardar um valor lido de **stdin**
- O número de valores a ler é variável e é ditado pela *string* de formatação
- Em caso de sucesso, a função **devolve** o número de variáveis da lista de argumentos preenchidas corretamente com valores lidos

Entrada formatada: scanf () (2)

➤ Exemplo: ler um valor **int** e um valor **double**

```
#include <stdio.h>
int main(void) // 01-scanf.c
{
    int    i;
    double d;

    printf("Digite um valor inteiro: ");
    scanf("%i", &i); // operador & devolve a posição de memória de i

    printf("Digite um valor real: ");
    scanf("%lf", &d);
    printf("Introduziu %i e %f\n", i, d);

    return 0;
}
```

- Contextualização da linguagem C
- Características mais relevantes da linguagem C
- Estrutura dum programa em C
- Tipos de dados primários do C
- Programar em C
- Valores e variáveis
- Operadores e expressões
- Entrada e saída normalizadas
- **C89 e C99**

➤ As versões do C:

- C Kernighan&Ritchie
 - C89 (ou C90)
 - C99
- } normas ISO

➤ Nesta UC vamos usar C99

➤ Na sua maioria, o C99 acrescenta características ou clarifica aspetos das versões anteriores, não invalida o código das versões anteriores

Algumas diferenças entre C89 e C99 ⁽¹⁾

1. São permitidos comentários ao estilo do C++ (`//`)
2. Está disponível o tipo `_Bool`
3. Foram adicionadas funções à biblioteca e criados alguns ficheiros *header*
4. São permitidos *arrays* de tamanho variável
5. As declarações de variáveis podem aparecer em qualquer parte dum bloco de código
6. Podem declarar-se variáveis nos ciclos `for`
7. São suportados caracteres não-ASCII (carateres alargados)

A cinzento = característica normalmente suportada pelo gcc ao compilar C89, a menos que o compilador esteja em modo *strict*

- 8. Foi introduzido um novo tipo de dados inteiro: **long long**
- 9. As funções devem declarar um valor a devolver
- 10. As macros podem ter um número variável de argumentos, representados por “...”
- 11. As funções podem ser qualificadas de **inline**
- 12. É permitido qualificar os apontadores como **restricted**, para evitar sobreposição das zonas de memória que referenciam

A cinzento = característica normalmente suportada pelo gcc ao compilar C89, a menos que o compilador esteja em modo *strict*